

정보처리기사 필기 실전 모의고사

정답과 해설 포함본 | 합격 기준: 과목당 40점 이상, 평균 60점 이상

1과목 소프트웨어 설계

1 보엠(Boehm)이 제안한 것으로, 개발 과정을 여러 번 반복하면서 매 반복마다 위험을 분석·해결해 나가는 생명주기 모델은? 생명주기

- ① 폭포수 모델
- ② 프로토타입 모델
- ③ 나선형 모델
- ④ V-모델

정답 ③

정답 근거 나선형(Spiral)의 한 사이클은 **계획 → 위험 분석 → 개발 → 고객 평가**로 돌고, 이 나선을 돌 때마다 시스템이 커집니다. 제안자가 보엠이라는 것과 '위험 분석'이 짝을 이룹니다.

함정 ②프로토타입도 '반복'해서 헛갈립니다. 하지만 프로토타입이 반복하는 목적은 **요구사항이 맞는지 확인**이고, 나선형이 반복하는 목적은 **위험을 줄이는 것**입니다. ①폭포수는 되돌아가지 않는 순차 모델이라 '반복'과 정반대입니다.

강사 한마디 저는 이 문제를 **목적어 읽기**로 풀니다. '무엇을 반복하는가'만 보세요. 위험을 반복하면 나선형, 견본을 반복하면 프로토타입입니다. 그리고 나선형은 **대규모·고비용 사업**과 세트로 나옵니다 — 위험 분석에 돈과 시간이 들기 때문이죠. 보기에 '대규모'가 같이 있으면 거의 확정입니다.

2 다음 요구사항 중 성격이 다른 하나는? 요구사항

- ① 관리자는 회원 정보를 수정할 수 있어야 한다
- ② 사용자는 상품을 검색할 수 있어야 한다
- ③ 시스템은 동시 접속자 1,000명을 처리해야 한다
- ④ 고객은 주문 내역을 조회할 수 있어야 한다

정답 ③

정답 근거 ①②④는 시스템이 **무엇을 하는가**(기능 요구사항)이고, ③은 **얼마나 잘 하는가**(성능=비기능 요구사항)입니다.

함정 '~해야 한다'는 표현이 모두 붙어 있어 문장 끝만 보면 구분이 안 됩니다. 출제자가 노린 지점이 정확히 여기입니다.

강사 한마디 문장 끝이 아니라 **동사**를 보세요. '수정한다·검색한다·조회한다'처럼 **사용자가 하는 행위**면 기능입니다. 반대로 '처리한다·응답한다·유지한다'에 **숫자나 품질 기준**이 붙으면 비기능이네요. 비기능은 결국 **성능·보안·신뢰성·사용성·이식성** 다섯 갈래로 떨어집니다. 숫자가 보이면 일단 비기능을 의심하세요.

3 요구공학 프로세스를 순서대로 바르게 나열한 것은? 요구공학

- ① 요구사항 도출 → 분석 → 명세 → 확인
- ② 요구사항 분석 → 도출 → 명세 → 확인
- ③ 요구사항 명세 → 도출 → 분석 → 확인
- ④ 요구사항 도출 → 명세 → 분석 → 확인

정답 ①

정답 근거 도출(Elicitation) → 분석(Analysis) → 명세(Specification) → 확인(Validation). 이해관계자에게서 끌어내고, 충돌·중복을 정리하고, 문서로 적고, 제대로 적혔는지 검토합니다.

함정 ④처럼 '명세'와 '분석'을 바꾼 보기가 자주 나옵니다. 분석하지 않은 요구를 문서로 쓸 수는 없으니 명세가 분석보다 앞설 수 없습니다.

강사 한마디 순서 문제는 **인과로 묶으면** 절대 안 틀립니다. 재료를 모으고(도출) → 다듬고(분석) → 계약서를 쓰고(명세) → 서명 전 검토(확인). 실무 흐름 그대로예요. 참고로 확인 단계 산출물이 **요구사항 명세서 검토·베이스라인**이고, 여기서 확정된 것이 이후 형상관리의 기준선이 됩니다.

4 스크럼(Scrum)에 대한 설명으로 옳지 않은 것은? 애자일

- ① 스프린트는 보통 2~4주 단위의 반복 개발 주기이다
- ② 제품 백로그는 우선순위가 매겨진 요구사항 목록이다
- ③ **스크럼 마스터는 팀원에게 업무를 지시하고 일정을 통제하는 관리자이다**
- ④ 번다운 차트는 남은 작업량의 추이를 보여준다

정답 ③

정답 근거 스크럼 마스터는 지시자가 아니라 **장애물을 제거하고 프로세스가 지켜지도록 돕는 조력자**(퍼실리테이터)입니다. 일을 지시하는 관리자가 아닙니다.

함정 '마스터'라는 단어 때문에 상급자로 착각하기 쉽습니다. 이 명칭이 곧 함정입니다.

강사 한마디 애자일 문제는 **수직적 통제를 말하면 오답**이라는 원칙 하나로 대부분 걸러집니다. 지시·통제·감독·상세 문서화가 등장하면 애자일 정신과 어긋납니다. 참고로 개발 항목의 우선순위를 정하는 사람은 **제품 책임자(PO)**이고, 팀은 자기조직적으로 일을 가져갑니다. '누가 시키느냐'가 아니라 '누가 돕느냐'가 스크럼 마스터입니다.

5 XP(eXtreme Programming)의 실천 방법으로 거리가 먼 것은? 애자일

- ① 짝 프로그래밍(Pair Programming)
- ② 테스트 주도 개발(TDD)
- ③ 지속적 통합(Continuous Integration)

④ 상세 설계 문서 우선 작성

정답 ④

정답 근거 XP는 문서보다 **동작하는 코드와 피드백**을 중시합니다. 상세 설계 문서를 먼저 완성하는 방식은 폭포수적 접근입니다.

함정 ④가 '좋은 개발 습관'처럼 보여 정답으로 고르기 망설여집니다. 하지만 문제는 '옳은가'가 아니라 'XP의 실천법인가'를 묻습니다.

강사 한마디 이런 문제에서는 **질문의 주어**를 반드시 확인하세요. 일반적으로 바람직한 것과 특정 방법론의 실천법은 다릅니다. XP의 다섯 가치(**용기·단순성·의사소통·피드백·존중**)를 떠올려 보면 '문서 우선'은 어디에도 걸리지 않습니다. 참고로 XP 실천법에는 리팩토링·소규모 릴리스·공동 코드 소유·계획 게임도 포함됩니다.

6 UML 다이어그램 중 구조(정적) 다이어그램에 해당하는 것으로만 묶인 것은? UML

- ① 클래스, 컴포넌트, 배치
- ② 유스케이스, 시퀀스, 활동
- ③ 클래스, 시퀀스, 상태
- ④ 활동, 배치, 커뮤니케이션

정답 ①

정답 근거 구조 다이어그램은 **클래스·객체·컴포넌트·배치·복합체구조·패키지** 6종입니다. ①은 모두 여기에 속합니다.

함정 ③처럼 정답 후보(클래스)에 동적인 것(시퀀스·상태)을 섞어 놓는 방식이 가장 흔한 오답 설계입니다. 하나만 보고 고르면 걸립니다.

강사 한마디 묶음 문제는 **확실한 오답 하나를 찾아 그 보기를 통째로 버리는** 방식이 빠릅니다. 시퀀스·활동·상태·유스케이스는 '움직임'이라 무조건 동적이니, 이 넷 중 하나라도 들어간 보기는 즉시 제외하세요. 그러면 ①만 남습니다. 시간 아끼는 실전 요령입니다.

7 UML에서 '주문' 객체가 사라지면 '주문상세' 객체도 함께 소멸하는 관계를 표현할 때 사용하는 것은? UML

- ① 연관(Association)
- ② 집합(Aggregation)
- ③ **합성(Composition)**
- ④ 의존(Dependency)

정답 ③

정답 근거 생명주기를 공유하는 강한 전체-부분 관계는 **합성(Composition)**이며 **속이 채워진 마름모(◆)**로 그립니다.

함정 ②집합(Aggregation)도 전체-부분이라 헛갈립니다. 차이는 딱 하나 — 집합은 부분이 **혼자서도 존재**합니다(부서가 없어져도 사원은 남음). 기호도 **빈 마름모(◇)**로 다릅니다.

강사 한마디 제 판단 기준은 **'전체를 지우면 부분도 지워지는가'** 이 한 문장입니다. 주문이 취소되면 주문상세는 존재할 이유가 없죠 → 합성. 반면 학과가 폐지돼도 학생은 다른 과로 갑니다 → 집합. 지문에 '함께 소멸·생명주기를 같이한다'가 나오면 **무조건 합성**입니다.

8 유스케이스 다이어그램에서 여러 유스케이스가 공통으로 반드시 포함하는 기능을 분리해 표현할 때 사용하는 관계는?

UML

- ① **include**
- ② extend
- ③ generalization
- ④ association

정답 ①

정답 근거 **include**는 기본 유스케이스가 실행될 때 **항상 반드시** 포함되어 실행되는 필수 관계입니다. '로그인 확인'처럼 여러 기능이 공통으로 필요로 하는 절차를 뽑아낼 때 씁니다.

함정 ②extend와 반드시 구분해야 합니다. extend는 **특정 조건에서만 선택적으로** 확장되는 관계입니다(예: 결제 시 '쿠폰 적용').

강사 한마디 한 글자로 외우면 **include는 '필수', extend는 '선택'**입니다. 실전에서는 지문에 '반드시·항상·공통으로'가 있으면 include, '~인 경우에·필요하면·추가로'가 있으면 extend로 갈리니 부사만 봐도 됩니다. 화살표 방향까지 묻는 문제도 있는데, 둘 다 점선 화살표이고 include는 포함하는 쪽에서 포함되는 쪽을 가리킵니다.

9 사용자 인터페이스 설계 원칙 중 유연성에 대한 설명으로 옳은 것은? UI

- ① 누구나 쉽게 이해하고 사용할 수 있어야 한다
- ② **사용자의 요구를 최대한 수용하고 실수를 방지할 수 있어야 한다**
- ③ 사용자의 목적을 정확하고 완전하게 달성해야 한다
- ④ 사용 방법을 쉽게 배우고 익힐 수 있어야 한다

정답 ②

정답 근거 유연성은 **사용자 요구 수용 + 오류 방지·복구**입니다. 되돌리기(Undo)·확인창·다양한 입력 경로가 여기 해당합니다.

함정 ①은 직관성, ③은 유효성, ④는 학습성입니다. 넷 다 그럴듯해서 정의를 정확히 기억하지 않으면 찍게 됩니다.

강사 한마디 이 네 개는 **키워드 한 단어씩만** 붙잡으면 끝납니다. 직관성=**이해**, 유효성=**목적 달성**, 학습성=**배우기**, 유연성=**수용·실수 방지**. 특히 '실수·오류'라는 단어가 보이면 유연성으로 직행하세요. 시험에서 이 네 개는 서로의 오답으로만 출제되므로, 넷을 함께 외우는 게 효율적입니다.

10 UI 설계 산출물을 완성도가 낮은 것부터 높은 순으로 바르게 나열한 것은? UI

- ① **와이어프레임 → 목업 → 프로토타입 → 스토리보드**
- ② 목업 → 와이어프레임 → 스토리보드 → 프로토타입
- ③ 프로토타입 → 목업 → 와이어프레임 → 스토리보드
- ④ 스토리보드 → 와이어프레임 → 목업 → 프로토타입

정답 ①

정답 근거 뼈대만 그린 **와이어프레임** → 실제처럼 보이지만 동작 안 하는 **목업** → 실제로 동작하는 **프로토타입** → 화면과 설명·흐름을 모두 묶은 최종 문서 **스토리보드** 순입니다.

함정 목업과 프로토타입의 선후를 바꾼 보기가 단골입니다. 판단 기준은 **동작 여부** 하나입니다.

강사 한마디 저는 '집 짓기'로 설명합니다. 도면 스케치(와이어프레임) → 모델하우스(목업, 보기만 됨) → 실제 시운전 되는 건물주택(프로토타입) → 시공 지침서 전체(스토리보드). **목업은 사진, 프로토타입은 영상**이라고 생각해도 좋습니다. 실기에서도 나오는 개념이라 지금 확실히 잡아두면 두 번 이득입니다.

11 모듈 A가 모듈 B를 호출하면서 B의 내부 실행 흐름을 결정하는 플래그 값을 전달하고 있다. 이때의 결합도는? **결합**

도

- ① 자료 결합도
- ② 스탬프 결합도
- ③ 제어 결합도
- ④ 공통 결합도

정답 ③

정답 근거 넘기는 값이 데이터가 아니라 '어떻게 동작하라'는 지시(제어 신호·플래그)이면 제어 결합도입니다. 호출하는 쪽이 상대의 내부 로직을 알고 있다는 뜻이라 결합이 강해집니다.

함정 '값을 전달한다'는 표현 때문에 ①자료 결합도로 착각하기 쉽습니다. 하지만 자료 결합은 순수한 데이터만 넘길 때이고, ②스탬프는 자료구조 전체를 넘기고 일부만 쓸 때입니다.

강사 한마디 이 문제는 단어 암기가 아니라 **상황 판별**을 요구하는 최근 출제 스타일입니다. 판단 순서를 이렇게 잡으세요 — 전역변수를 쓰나?(공통) → 내부를 직접 건드리나?(내용) → 넘기는 게 지시인가?(제어) → 구조체 통째인가?(스탬프) → 아니면 자료. 이 다섯 질문이면 어떤 시나리오든 분류됩니다.

12 하나의 모듈 안에 '초기화', '로그 기록', '메모리 해제'처럼 서로 관련은 없지만 프로그램 시작 시점에 함께 수행되는 기능들이 모여 있다. 이 모듈의 응집도는? **응집도**

- ① 기능적 응집도
- ② 시간적 응집도
- ③ 논리적 응집도
- ④ 우연적 응집도

정답 ②

정답 근거 기능들이 **특정 시점(시간대)에 함께 실행된다**는 이유만으로 묶었다면 시간적 응집도입니다. 초기화 루틴이 대표적인 예입니다.

함정 ④우연적으로 보기 쉽습니다. 하지만 우연적은 **아무 관련도, 아무 기준도 없이** 모인 경우입니다. 여기서는 '시작 시점'이라는 분명한 기준이 있으므로 우연적이지 않습니다. ③논리적은 '비슷한 성격끼리'(예: 모든 입력 처리 모음) 묶은 경우입니다.

강사 한마디 응집도 판별의 핵심은 **'무엇을 기준으로 묶었나'**를 찾는 것입니다. 시점이면 시간적, 성격이면 논리적, 실행 순서면 절차적, 같은 데이터면 교환적, 출력이 다음 입력이면 순차적, 단일 기능이면 기능적. 기준이 아예 없으면 우연적이고요. 지문에 '함께 수행·같은 시점'이 보이면 시간적입니다.

13 팬인(Fan-In)과 팬아웃(Fan-Out)에 대한 설명으로 옳은 것은? 모듈화

- ① 팬인이 높으면 재사용성이 높고, 팬아웃이 높으면 복잡도가 증가한다
- ② 팬인이 높으면 복잡도가 증가하고, 팬아웃이 높으면 재사용성이 높다
- ③ 팬인과 팬아웃 모두 높을수록 좋은 설계이다
- ④ 팬인과 팬아웃 모두 낮을수록 좋은 설계이다

정답 ①

정답 근거 팬인은 나를 호출하는 상위 모듈 수라 높을수록 여러 곳에서 쓰이는 **공통·재사용 모듈**임을 뜻합니다. 팬아웃은 내가 호출하는 하위 모듈 수라 높을수록 관리할 것이 많아 **복잡**해집니다.

함정 ②는 정확히 반대로 서술한 보기입니다. 방향(들어오는 화살표 vs 나가는 화살표)을 반대로 기억하면 그대로 걸립니다.

강사 한마디 화살표로 그려서 기억하세요. **In**은 **들어오는 것**이니 나를 찾는 사람이 많다는 뜻 = 인기 있는 공통 모듈 = 좋음. **Out**은 **나가는 것**이니 내가 챙길 부하가 많다는 뜻 = 부담 = 나쁨. 설계 목표는 한 줄로 '**팬인은 높게, 팬아웃은 낮게**'입니다. 결합도·응집도와 함께 물으면 '결합도 낮게, 응집도 높게, 팬인 높게, 팬아웃 낮게'가 정답 세트입니다.

14 데이터를 처리하는 여러 단계를 거치며, 각 단계의 출력이 다음 단계의 입력이 되는 구조로 컴파일러나 스트림 처리에 적합한 아키텍처 패턴은? 아키텍처

- ① MVC 패턴
- ② 계층화 패턴
- ③ **파이프-필터 패턴**
- ④ 브로커 패턴

정답 ③

정답 근거 각 **필터**가 데이터를 변환하고 **파이프**를 통해 다음 필터로 흘러보내는 구조가 **파이프-필터**입니다. 유닉스 셸의 `|`나 컴파일러 단계가 대표 예입니다.

함정 ②계층화도 '단계'라는 말 때문에 헷갈립니다. 하지만 계층화는 **추상화 수준별로 층을 나눈 것**(표현-업무-데이터)이지, 데이터가 변환되며 흘러가는 구조가 아닙니다.

강사 한마디 구분 기준은 '**데이터가 흐르는가, 역할이 나뉘는가**'입니다. 데이터가 변형되며 통과하면 파이프-필터, 책임을 층으로 쌓았으면 계층화, 화면·데이터·제어로 나뉘면 MVC입니다. 아키텍처 패턴은 종류가 많지만 시험은 **MVC·계층화·파이프필터·클라이언트서버·브로커·마스터슬레이브** 정도만 반복 출제합니다. 이 여섯 개만 확실히 하세요.

15 GoF 디자인 패턴 중 구조(Structural) 패턴에 속하지 않는 것은? 디자인패턴

- ① 어댑터(Adapter)
- ② 데코레이터(Decorator)
- ③ 퍼사드(Facade)
- ④ 옵서버(Observer)

정답 ④

정답 근거 옵서버는 객체 간 상호작용·책임을 다루는 행위(Behavioral) 패턴입니다. 나머지 셋은 객체를 조립·연결하는 구조 패턴입니다.

함정 패턴 이름은 아는데 분류를 모르면 못 푸는 문제입니다. 최근에는 이렇게 '분류'를 묻는 형태가 이름 자체를 묻는 것보다 많습니다.

강사 한마디 23개를 다 외우려 하지 마세요. 분류당 대표 2~3개만 확실히 하면 대부분 풀립니다. 생성=싱글턴·팩토리·빌더, 구조=어댑터·데코레이터·퍼사드·프록시, 행위=옵서버·스트래티지·템플릿메서드·커맨드·이터레이터. 요령 하나 — 이름이 '~하는 사람/행동' 느낌이면(관찰자·전략·명령·반복자) 대개 행위 패턴입니다. 실제로 잘 맞습니다.

16 기존 코드를 수정하지 않고 객체에 기능을 동적으로 덧붙이고 싶을 때 사용하기에 가장 적절한 패턴은? 디자인패턴

- ① 싱글턴(Singleton)
- ② 데코레이터(Decorator)
- ③ 어댑터(Adapter)
- ④ 팩토리 메서드(Factory Method)

정답 ②

정답 근거 데코레이터는 객체를 같은 인터페이스의 껍질로 계속 감싸며 기능을 층층이 추가합니다. 상속 없이 런타임에 조합할 수 있는 것이 장점입니다.

함정 ③어댑터와 자주 혼동됩니다. 어댑터는 인터페이스가 안 맞는 것을 맞춰 주는 변환기이지 기능을 더하지 않습니다. 목적이 '호환'이나 '추가'냐가 갈림길입니다.

강사 한마디 이름 그대로 생각하면 쉽습니다. 데코(장식)는 케이크에 장식을 계속 얹는 것 — 본체는 그대로인데 기능이 늘죠. 어댑터는 해외여행 갈 때 쓰는 돼지코 변환 플러그입니다 — 전기를 더 주는 게 아니라 모양만 맞춰 줍니다. 이 두 비유면 헷갈릴 일이 없습니다.

17 데이터베이스 연결 객체처럼 시스템에서 단 하나의 인스턴스만 생성하여 전역적으로 공유해야 할 때 사용하는 패턴은? 디자인패턴

- ① 프로토타입(Prototype)
- ② 빌더(Builder)
- ③ 싱글턴(Singleton)
- ④ 브리지(Bridge)

정답 ③

정답 근거 싱글턴은 생성자를 감추고 정적 메서드로만 인스턴스를 제공해 **객체가 하나만** 존재하도록 보장합니다. 설정·로그·커넥션 풀이 대표 사례입니다.

함정 ①프로토타입도 생성 패턴이라 후보로 보이지만, 이건 기존 객체를 **복제**해서 새로 만드는 패턴이라 '하나만'과 정반대입니다.

강사 한마디 싱글턴은 시험에서 **생성 패턴 중 최빈출**입니다. '단 하나·유일한·전역 공유'라는 표현이 나오면 고민 없이 고르세요. 한 가지 덧붙이면, 실무에서는 싱글턴이 **테스트를 어렵게 하고 전역 상태를 만든다**는 이유로 남용을 경계합니다. 이런 단점을 묻는 문제도 가끔 나오니 '남용하면 결합도가 올라간다'는 점도 기억해 두세요.

18 기업 내 여러 시스템을 통합할 때, 시스템들을 1:1로 직접 연결하여 초기에는 단순하지만 시스템 수가 늘면 연결선이 급격히 복잡해지는 EAI 구축 유형은? 인터페이스

- ① Point-to-Point
- ② Hub & Spoke
- ③ Message Bus
- ④ Hybrid

정답 ①

정답 근거 **Point-to-Point**는 필요한 시스템끼리 직접 연결하는 가장 단순한 방식입니다. 시스템이 n개면 연결이 최대 $n(n-1)/2$ 개까지 늘어 거미줄처럼 됩니다.

함정 ②Hub & Spoke는 중앙 허브를 두는 방식이라 오히려 연결 수가 **n개**로 줄어듭니다. '단순하다'는 서술만 보고 고르면 헷갈립니다.

강사 한마디 네 유형은 **그림으로** 외우는 게 최고입니다. P2P는 **거미줄**, Hub&Spoke는 **자전거 바퀴살**, Message Bus는 **지하철 노선**, Hybrid는 둘의 혼합. 그리고 각 단점도 세트로 — P2P는 확장성 최악, Hub&Spoke는 **허브가 죽으면 전체 마비**(단일 장애점). 이 '허브 단일 장애점'이 자주 출제됩니다.

19 미들웨어에 대한 설명으로 옳지 않은 것은? 미들웨어

- ① RPC는 원격에 있는 프로시저를 로컬처럼 호출하게 해 준다
- ② MOM은 메시지 기반의 비동기 통신을 지원한다
- ③ TP 모니터는 온라인 트랜잭션을 처리·감시한다
- ④ **WAS는 정적인 HTML 파일만 전달하는 웹 서버를 의미한다**

정답 ④

정답 근거 WAS(Web Application Server)는 JSP·서블릿 등을 실행해 **동적인 결과**를 만들어 내는 서버입니다. 정적 파일만 전달하는 것은 일반 **웹 서버**(Apache, Nginx)의 역할입니다.

함정 WAS와 웹 서버를 같은 것으로 알고 있으면 그냥 넘어갑니다. 실제로 둘을 섞어 서술하는 오답이 자주 등장합니다.

강사 한마디 웹 서버는 배달, WAS는 조리라고 기억하세요. 이미 만들어진 파일을 그대로 주면 웹 서버, 요청을 받아 계산해서 만들어 주면 WAS입니다. 미들웨어 문제는 대부분 이런 '옳지 않은 것' 형태로 나오는데, **RPC=원격 호출, MOM=메시지·비동기, TP모니터=거래, ORB=객체**만 정확하면 나머지 하나가 자동으로 답이 됩니다.

20 시스템 간 데이터 교환 형식에 대한 설명으로 옳은 것은? 인터페이스

- ① JSON은 태그를 사용하여 데이터를 표현하는 마크업 언어이다
- ② **XML은 사용자 정의 태그를 사용해 구조화된 데이터를 표현한다**
- ③ YAML은 이진 형식으로만 데이터를 표현한다
- ④ JSON은 XML보다 항상 파일 크기가 크다

정답 ②

정답 근거 XML은 사용자가 직접 태그를 정의해 데이터를 구조화하는 마크업 언어입니다. 문서 구조를 엄격히 표현할 수 있어 표준 연계에 오래 쓰였습니다.

함정 ①은 JSON과 XML의 설명을 바꿔 놓았습니다. JSON은 태그가 아니라 **키:값 쌍과 중괄호**로 표현합니다. ④도 반대입니다 — JSON이 태그 반복이 없어 보통 **더 가볍습니다**.

강사 한마디 셋의 성격을 한 줄로 잡아두세요. **XML은 무겁지만 엄격, JSON은 가볍고 웹 표준, YAML은 사람이 읽기 편한 설정 파일용**(들여쓰기로 구조 표현, 텍스트 기반입니다). 요즘 시스템 연계는 대부분 JSON이라 '가볍다·경량'이라는 서술이 JSON에 붙으면 옳은 문장입니다.

2과목 소프트웨어 개발

21 자료구조에 대한 설명으로 옳지 않은 것은? 자료구조

- ① 스택은 LIFO 방식으로 동작하며 함수의 재귀 호출에 사용된다
- ② 큐는 FIFO 방식으로 동작하며 프린터 대기열에 사용된다
- ③ 데크(Deque)는 양쪽 끝에서 삽입과 삭제가 모두 가능하다
- ④ 트리는 노드가 한 줄로 연결된 선형 자료구조이다

정답 ④

정답 근거 트리는 부모-자식 계층 구조를 갖는 비선형 자료구조입니다. 그래프도 마찬가지로 비선형입니다.

함정 ①②③이 모두 맞는 설명이라 끝까지 읽지 않으면 앞에서 답을 골라 버립니다. '옳지 않은 것'을 묻는 문제는 반드시 4번까지 읽어야 합니다.

강사 한마디 선형/비선형 구분은 '한 줄로 세울 수 있는가'로 판단하세요. 배열·연결리스트·스택·큐·데크는 한 줄(선형), 트리·그래프는 가지가 갈라져 한 줄이 안 됩니다(비선형). 참고로 이 문제처럼 **2025년 1회에 자료구조가 대거 출제**돼 난이도를 올렸습니다. 2과목에서는 자료구조를 절대 버리지 마세요.

22 스택에 다음 순서로 연산을 수행했을 때, 최종적으로 스택에 남아 있는 데이터를 아래에서 위로 나열한 것은? 스택

```
push(1), push(2), push(3), pop(), push(4), pop(), push(5)
```

- ① 1, 2, 5
- ② 1, 2, 4
- ③ 1, 3, 5
- ④ 2, 4, 5

정답 ①

정답 근거 push 1,2,3 → 스택 [1,2,3]. pop()으로 **3 제거** → [1,2]. push 4 → [1,2,4]. pop()으로 **4 제거** → [1,2]. push 5 → **[1,2,5]**.

함정 ②는 마지막 pop이 없었다고 착각한 경우, ③은 pop을 앞에서만 했다고 본 경우입니다. 연산이 섞여 있으면 **순서대로 한 줄씩** 적어야 합니다.

강사 한마디 이런 추적 문제는 머릿속으로 하면 90% 틀립니다. 반드시 **옆에 스택 상태를 손으로 적으세요**. CBT라 종이가 없다고 걱정할 필요 없습니다 — 시험장에서 **연습장(백지)과 필기구를 줍니다**. pop은 항상 맨 위(마지막에 넣은 것)를 꺼낸다는 것만 지키면 실수할 여지가 없습니다.

23 후위 표기법(Postfix)으로 표현된 다음 수식의 계산 결과는? 후위표기

$3\ 4\ +\ 2\ * \ 7\ -$

- ① 7
- ② 3
- ③ 10
- ④ 21

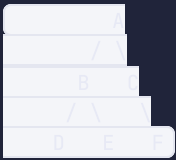
정답 ①

정답 근거 왼쪽부터 읽어 피연산자는 쌓고, 연산자를 만나면 **스택에서 두 개를 꺼내 계산**합니다. $3\ 4\ + \rightarrow 7$, $7\ 2\ * \rightarrow 14$, $14\ 7\ - \rightarrow 7$.

함정 연산 순서를 중위식처럼 $3+4*2-7$ 로 착각하면 4가 나옵니다. 후위 표기는 **연산자가 나온 시점에 바로 계산**합니다.

강사 한마디 후위 표기는 **스택 응용의 대표 문제**라 거의 매 회차 나옵니다. 계산 요령은 딱 하나 — 연산자를 만나면 **바로 앞 두 개**를 묶어 계산하고 그 자리에 결과를 써넣기. 뿔셈·나눗셈은 **순서가 중요합니다**(먼저 꺼낸 것이 뒤쪽 피연산자). $14\ 7\ -$ 는 $7-14$ 가 아니라 **$14-7$** 입니다. 이 순서에서 실수가 제일 많이 납니다.

24 다음 이진 트리를 중위 순회(Inorder)한 결과로 옳은 것은? 트리



- ① **D B E A C F**
- ② A B D E C F
- ③ D E B F C A
- ④ D B E A F C

정답 ①

정답 근거 중위는 **왼쪽 → 뿌리 → 오른쪽**입니다. B의 서브트리를 먼저 중위로($D \rightarrow B \rightarrow E$), 다음 뿌리 A, 마지막 C의 서브트리(C에는 오른쪽 자식 F만 있으므로 $C \rightarrow F$). 따라서 **D B E A C F**.

함정 ④는 마지막 C-F 순서를 뒤집은 것입니다. F는 C의 **오른쪽** 자식이므로 중위에서는 C가 먼저입니다. ②는 전위, ③은 후위 결과입니다.

강사 한마디 순회 문제는 **재귀적으로** 접근하세요. '전체 = 왼쪽 뭉치 + 뿌리 + 오른쪽 뭉치'로 큰 덩어리를 먼저 나눈 다음 각 덩어리 안에서 같은 규칙을 반복합니다. 그리고 팁 하나 — **중위 순회는 이진 탐색 트리에서 오름차순 정렬 결과**가 됩니다. 이 성질을 묻는 응용 문제도 나오니 함께 기억하세요.

25 노드가 15개인 이진 트리에서 단말 노드(leaf)가 8개일 때, 자식이 2개인 노드의 개수는? 트리

- ① 6개
- ② 7개
- ③ 8개
- ④ 9개

정답 ②

정답 근거 이진 트리에서는 항상 **[단말 노드 수] = [자식이 2개인 노드 수] + 1**이 성립합니다. 따라서 자식이 2개인 노드는 $8 - 1 = 7$ 개입니다.

함정 전체 노드 15개에서 단말 8개를 뺀 7을 그냥 답으로 고르면 **우연히 맞습니다**. 하지만 그건 자식이 1개인 노드가 0개일 때만 성립하는 우연이라, 숫자가 바뀌면 틀립니다.

강사 한마디 공식 $n_0 = n_2 + 1$ 은 외워 두면 좋습니다. 증명은 필요 없고 관계만 기억하세요. 이 문제는 최근 자료구조 강화 경향에서 나오는 **계산형 트리 문제**의 전형입니다. 함께 나오는 공식으로 **높이 h인 포화 이진 트리의 노드 수 = $2^h - 1$, 레벨 k의 최대 노드 수 = $2^{(k-1)}$** 도 챙겨 두세요.

26 다음 데이터를 버블 정렬로 오름차순 정렬할 때, 1회전(1 pass)을 마친 결과는? 정렬

8, 5, 6, 2, 4

- ① 5, 6, 2, 4, 8
- ② 2, 5, 6, 8, 4
- ③ 5, 8, 6, 2, 4
- ④ 2, 4, 5, 6, 8

정답 ①

정답 근거 버블은 **인접한 두 개**를 비교·교환하며 끝까지 훑습니다. (8,5)→교환 [5,8,6,2,4], (8,6)→교환 [5,6,8,2,4], (8,2)→교환 [5,6,2,8,4], (8,4)→교환 **[5,6,2,4,8]**. 1회전이 끝나면 **가장 큰 값 8이 맨 뒤에** 자리잡습니다.

함정 ②는 **선택 정렬**의 1회전 결과입니다(최솟값 2를 맨 앞으로). 두 정렬의 1회전 결과를 나란히 보기로 주는 것이 전형적인 함정입니다.

강사 한마디 1회전 결과만 봐도 어떤 정렬인지 구분됩니다 — **버블은 최대값이 맨 뒤로, 선택은 최솟값이 맨 앞으로, 삽입은 앞 두 개만 정렬**됩니다. 이 세 가지 '지문(指紋)'을 기억하면 어떤 정렬의 몇 회전을 묻든 바로 답이 보입니다. 정렬 과정 추적은 2과목 단골이니 손으로 두세 번 굴려 보세요.

27 정렬 알고리즘의 시간 복잡도에 대한 설명으로 옳은 것은? 정렬

- ① 퀵 정렬은 최악의 경우에도 $O(n \log n)$ 을 보장한다
- ② 병합 정렬은 평균과 최악 모두 $O(n \log n)$ 이다
- ③ 선택 정렬의 평균 시간 복잡도는 $O(n \log n)$ 이다
- ④ 버블 정렬은 이미 정렬된 경우 $O(\log n)$ 이다

정답 ②

정답 근거 병합 정렬은 항상 절반씩 나눠 합치므로 데이터 상태와 무관하게 **평균·최악 모두 $O(n \log n)$** 입니다. 안정 정렬이기도 합니다.

함정 ①이 가장 매력적인 오답입니다. 퀵 정렬은 평균은 $O(n \log n)$ 이지만 피벗이 한쪽으로 치우치면 **최악 $O(n^2)$** 입니다. '퀵=제일 빠름'이라는 인상 때문에 최악까지 보장한다고 착각하기 쉽습니다.

강사 한마디 복잡도 문제는 **퀵의 최악만 정확히 알면** 절반이 풀립니다. 정리하면 — 느린 3형제(버블·선택·삽입) 평균 $O(n^2)$, 빠른 3형제(퀵·병합·힙) 평균 $O(n \log n)$, 그중 **퀵만 최악 $O(n^2)$** . 삽입 정렬은 이미 정렬된 데이터에서 **최선 $O(n)$** 이 되는 예외가 있으니 이것도 함께 알아두면 좋습니다.

28 1부터 100까지 정렬된 100개의 데이터에서 이진 검색으로 특정 값을 찾을 때, 최악의 경우 비교 횟수는? 검색

- ① 5회
- ② 6회
- ③ **7회**
- ④ 10회

정답 ③

정답 근거 이진 검색은 한 번 비교마다 후보가 절반으로 줄어듭니다. $100 \rightarrow 50 \rightarrow 25 \rightarrow 13 \rightarrow 7 \rightarrow 4 \rightarrow 2 \rightarrow 1$ 로 **7번**이면 후보가 하나로 좁혀집니다($\lceil \log_2 101 \rceil = 7$).

함정 ④10회는 $\log_{10} 100 = 2$ 같은 계산 착오나, 순차 검색과 혼동한 경우입니다. 이진 검색의 기준은 **2의 거듭제곱**입니다.

강사 한마디 암산 요령을 드리면 **2의 거듭제곱을 외워두고 데이터 수를 넘기는 지점을** 찾으면 됩니다. $2^6=64$, $2^7=128$ 이므로 100은 64와 128 사이 → **7회**. 계산기 없이 3초면 끝납니다. 참고로 이진 검색은 **정렬이 전제**라는 조건이 함께 출제되니, 그 조건도 반드시 챙기세요.

29 해싱에서 서로 다른 키가 같은 버킷에 할당되는 충돌(Collision)을 해결하는 방법으로, 같은 주소에 연결 리스트를 이어 붙이는 방식은? 해싱

- ① 선형 조사법
- ② 이차 조사법
- ③ 체이닝(Chaining)
- ④ 이중 해싱

정답 ③

정답 근거 체이닝은 하나의 버킷에 연결 리스트를 매달아 충돌한 항목을 계속 이어 붙입니다. 버킷이 가득 차는 개념이 없어 적재율이 높아도 동작합니다.

참정 ①②④는 모두 개방 주소법(빈 자리를 찾아 옮겨 담는 방식)에 속합니다. 셋을 하나로 묶어 두면 헛갈릴 일이 없습니다.

강사 한마디 충돌 해결은 두 갈래로만 나누면 끝납니다 — 밖으로 매달면 체이닝, 안에서 빈 자리를 찾으면 개방 주소법. 개방 주소법의 세부(선형·이차·이중 해싱)는 이름만 알면 충분하고 구현까지는 안 나옵니다. 해싱의 강점은 평균 **O(1)** 검색이라는 점인데, 이건 복잡도 비교 문제에서 자주 물어봅니다.

30 소프트웨어 형상관리(SCM)의 활동에 해당하지 않는 것은? 형상관리

- ① 형상 식별
- ② 형상 통제
- ③ 형상 감사
- ④ 형상 배포

정답 ④

정답 근거 형상관리의 활동은 식별 → 통제 → 감사 → 기록(상태 보고) 네 가지입니다. '형상 배포'는 표준 활동 명칭이 아닙니다.

참정 '배포'는 소프트웨어 개발에서 늘 쓰는 익숙한 단어라 자연스럽게 넘어갑니다. 이런 문제는 없는 용어를 그럴듯하게 만들어 넣는 방식입니다.

강사 한마디 보기에 익숙하지만 목록에 없는 단어가 하나 섞여 있으면 그게 답인 경우가 많습니다. 정확한 네 개를 외우고 있어야 판별되니, '식·통·감·기'로 묶어 두세요. 형상관리의 목적도 함께 — 변경 이력 추적, 이전 버전 복구, 무단 변경 방지입니다.

31 버전 관리 도구에 대한 설명으로 옳은 것은? 버전관리

- ① SVN은 각 개발자가 전체 이력을 갖는 분산형이다
- ② Git에서 branch는 원격 저장소에 변경을 올리는 명령이다
- ③ **Git은 분산형으로, 로컬에서도 커밋 이력을 관리할 수 있다**
- ④ CVS는 Git보다 나중에 개발된 최신 도구이다

정답 ③

정답 근거 Git은 분산형(DVCS)이라 개발자마다 저장소 전체 복사본을 갖고, 네트워크 없이도 커밋할 수 있습니다.

함정 ①은 SVN과 Git의 설명이 바뀌었습니다(SVN은 중앙집중형). ②의 '원격에 올리는' 명령은 **push**이고, branch는 가지를 만드는 명령입니다.

강사 한마디 Git 명령은 **방향**으로 외우면 안 헛갈립니다. **push**는 밀어 올리기(로컬→원격), **pull**는 당겨 오기(원격→로컬), **commit**은 로컬에 **확정 저장**, **branch**는 **가지치기**, **merge**는 **합치기**. 여기에 **clone**(복제), **checkout**(전환)까지 일곱 개면 필기에서 물어보는 범위는 다 덮습니다.

32 소프트웨어 패키징에 대한 설명으로 옳지 않은 것은? 패키징

- ① 패키징은 개발자 관점이 아닌 사용자 관점에서 진행한다
- ② 릴리스 노트에는 버전, 변경 사항, 개선 내용 등을 기록한다
- ③ DRM은 콘텐츠의 불법 복제와 유통을 방지하는 저작권 보호 기술이다
- ④ **패키징은 소스 코드를 전부 공개하여 배포하는 것을 원칙으로 한다**

정답 ④

정답 근거 패키징은 사용자가 **설치·실행할 수 있는 형태로 묶는** 작업입니다. 소스 코드 전면 공개는 원칙이 아니며, 오히려 **DRM으로 보호**하는 것이 함께 다뤄집니다.

함정 ④는 오픈소스 개념과 섞어 만든 오답입니다. 패키징과 오픈소스 라이선스는 별개 주제입니다.

강사 한마디 패키징 주제는 **'사용자 관점'**이라는 한 문장이 거의 모든 문제의 열쇠입니다. 사용자 중심·설치 편의·매뉴얼 제공·저작권 보호가 옳은 방향이고, 개발 편의나 내부 공개를 말하면 오답입니다. 함께 나오는 산출물은 **릴리스 노트·설치 매뉴얼·사용자 매뉴얼** 세 가지입니다.

33 테스트 원리 중, 동일한 테스트 케이스를 반복하면 더 이상 새로운 결함을 발견하지 못하므로 테스트 케이스를 지속적으로 개선해야 한다는 것은? 테스트

- ① 결함 집중
- ② **살충제 패러독스**
- ③ 오류-부재의 궤변
- ④ 완벽한 테스트는 불가능

정답 ②

정답 근거 같은 살충제를 계속 쓰면 해충에 내성이 생기듯, 같은 테스트 케이스로는 새 결함이 안 잡힌다는 것이 **살충제 패러독스**입니다.

참정 ③오류-부재의 궤변은 **결함을 다 없앴어도 요구사항과 다르면 좋은 소프트웨어가 아니다**라는 뜻으로, 전혀 다른 원리입니다. 이름이 낯설어 서로 바꿔 내기 좋은 짝입니다.

강사 한마디 테스트 7원리 중 시험에 나오는 건 사실상 **네 개**입니다 — 살충제 패러독스(케이스 개선), 결함 집중(파레토 80:20), 오류-부재의 궤변(요구 불일치), 완벽한 테스트 불가능. 각각 **키워드 한 단어**로 붙잡아 두면 어떤 서술로 바꿔 내도 잡힙니다.

34 화이트박스 테스트에서 사용하는 검증 기준(커버리지)이 아닌 것은? 테스트

- ① 구문(문장) 커버리지
- ② 결정(분기) 커버리지
- ③ 조건 커버리지
- ④ **동등 분할 커버리지**

정답 ④

정답 근거 **동등 분할**은 입력 값을 그룹으로 나눠 대표값을 시험하는 **블랙박스** 기법입니다. 커버리지라는 개념 자체가 아닙니다.

참정 '커버리지'라는 단어를 뒤에 붙여 놓아 그럴듯해 보입니다. 앞 단어(동등 분할)가 블랙박스 기법이라는 걸 알아야 걸러집니다.

강사 한마디 두 기법군을 **완전히 분리해서** 외우세요. 화이트박스 = **커버리지 계열**(구문→결정→조건→조건/결정→다중조건, 뒤로 갈수록 강함) + 기초 경로. 블랙박스 = **동등분할·경계값분석·원인결과그래프·결정표·상태전이·오류추정**. 한쪽 목록에 있는 단어가 반대쪽에 섞여 나오면 그게 답입니다.

35 통합 테스트에서 상향식(Bottom-Up) 방식을 수행할 때 필요한 것은? 테스트

- ① 스텝(Stub)
- ② 드라이버(Driver)
- ③ 목 오브젝트만 사용
- ④ 별도의 보조 모듈이 필요 없다

정답 ②

정답 근거 상향식은 하위 모듈부터 검증하므로, 아직 없는 상위 모듈 역할을 대신할 드라이버가 필요합니다. 드라이버가 하위 모듈을 호출해 결과를 확인합니다.

함정 ①스텝은 하향식에서 필요한 가짜 하위 모듈입니다. 이 둘을 바꿔 내는 것이 2과목에서 가장 자주 나오는 함정입니다.

강사 한마디 저는 이렇게 외우게 합니다 - '위에서 아래로 가면 아래가 없으니 스텝, 아래에서 위로 가면 위가 없으니 드라이버'. 없는 쪽을 채운다는 관점이면 절대 안 헛갈립니다. 덧붙여 드라이버는 운전자(위에서 조종), 스텝은 그루터기(아래에 남은 밑동)라는 어원으로도 연결됩니다.

36 다음 제어 흐름 그래프의 순환 복잡도(McCabe)는? (노드 8개, 간선 11개) 테스트

- ① 3
- ② 4
- ③ 5
- ④ 6

정답 ③

정답 근거 $V(G) = E - N + 2 = 11 - 8 + 2 = 5$ 입니다. E는 간선(edge), N은 노드(node) 수입니다.

함정 E와 N을 바꿔 $8 - 11 + 2$ 로 계산하면 -1이 되어 당황하게 됩니다. 간선이 항상 앞입니다.

강사 한마디 공식을 못 외우겠으면 '복잡도는 양수여야 한다'로 감산하세요. 보통 간선이 노드보다 많으니 큰 수에서 작은 수를 빼고 2를 더하면 됩니다. 그리고 $V(G) = \text{판단 노드(조건문) 수} + 1$ 이라는 두 번째 공식도 있습니다. 그래프 대신 코드를 주고 if 개수를 세게 하는 문제가 나오면 이쪽이 훨씬 빠릅니다.

37 소프트웨어 테스트 레벨을 순서대로 바르게 나열한 것은? 테스트

- ① 단위 테스트 → 통합 테스트 → 시스템 테스트 → 인수 테스트
- ② 통합 테스트 → 단위 테스트 → 인수 테스트 → 시스템 테스트
- ③ 단위 테스트 → 시스템 테스트 → 통합 테스트 → 인수 테스트
- ④ 인수 테스트 → 시스템 테스트 → 통합 테스트 → 단위 테스트

정답 ①

정답 근거 단위(모듈 하나) → 통합(모듈 간 연결) → 시스템(전체) → 인수(고객 확인) 순으로 범위가 넓어집니다.

함정 ③처럼 시스템과 통합을 바꾼 보기가 혼합니다. 모듈을 합치지도 않고 전체 시스템을 시험할 수는 없습니다.

강사 한마디 V-모델을 떠올리면 순서가 저절로 나옵니다. 왼쪽 내려가는 축(요구분석→설계→상세설계→구현)의 거울상이 오른쪽 올라가는 테스트 축입니다. 상세설계↔단위, 설계↔통합, 요구분석↔시스템, 사용자요구↔인수. 이렇게 짝을 지어두면 순서 문제와 V-모델 문제를 한 번에 잡습니다.

38 블랙박스 테스트 기법 중, 입력 조건의 경계에 있는 값에서 결함이 발생할 확률이 높다는 점을 이용하는 것은? 테스트

- ① 동등 분할 검사
- ② 경계값 분석
- ③ 오류 예측 검사
- ④ 원인-결과 그래프 검사

정답 ②

정답 근거 경계값 분석은 유효 범위의 최솟값·최댓값과 그 바로 앞뒤 값을 집중적으로 시험합니다. 개발자가 < 와 ≤ 를 헷갈리는 지점이 바로 경계이기 때문입니다.

함정 ①동등 분할과 짝으로 출제됩니다. 동등 분할은 구간을 나눠 대표값 하나씩 뽑는 것이고, 경계값 분석은 그 구간의 가장자리를 뽑는 것입니다.

강사 한마디 실무 감각으로 이해하면 절대 안 잊습니다. '1~100만 입력 가능'이라는 조건이 있으면 버그는 50에서 나지 않고 0, 1, 100, 101에서 납니다. 그래서 경계값을 봅니다. 두 기법은 보통 함께 사용하며, 시험에서도 짝으로 나오니 세트로 외우세요.

39 소프트웨어의 외부 동작은 그대로 두고 내부 구조만 개선하여 가독성과 유지보수성을 높이는 활동은? 품질

① 리팩토링

- ② 리엔지니어링
- ③ 리버스 엔지니어링
- ④ 마이그레이션

정답 ①

정답 근거 리팩토링은 기능(외부 동작)을 바꾸지 않으면서 코드 구조를 정리하는 작업입니다.

함정 ②리엔지니어링은 기존 시스템을 재분석·재구축하는 더 큰 범위의 작업이고, ③리버스 엔지니어링은 완성품에서 설계를 역으로 추출하는 것입니다. 이름이 비슷해 헷갈릴 수 있습니다.

강사 한마디 're-' 접두어는 방향으로 구분하세요. 리팩토링=안을 정리(기능 불변), 리버스=거꾸로 설계 추출(코드→문서), 리엔지니어링=아예 새로 다시(재구축). 여기에 마이그레이션=옮기기(다른 환경으로 이전)까지 넣으면 네 개가 한 세트로 정리됩니다.

40 애플리케이션 성능 측정 지표에 해당하지 않는 것은? 성능

- ① 처리량(Throughput)
- ② 응답 시간(Response Time)
- ③ 자원 사용률(Resource Utilization)
- ④ 기능 점수(Function Point)

정답 ④

정답 근거 기능 점수(FP)는 소프트웨어의 기능 규모를 세어 개발 비용·공수를 산정하는 지표(5과목)이지, 실행 성능 지표가 아닙니다.

함정 '점수'라는 말이 측정 지표처럼 들릴 수 있습니다. 성능 지표 네 개를 정확히 알고 있어야 걸러집니다.

강사 한마디 성능 4대 지표는 처리량·응답 시간·경과 시간·자원 사용률입니다. 다 '실행 중에 잴 수 있는 것'이라는 공통점이 있어요. 반대로 LOC·기능점수·COCOMO는 만들기 전에 추정하는 것이라 성격이 다릅니다. '재는 것이냐 미리 추정하는 것이냐'로 나누면 이런 문제는 실수하지 않습니다.

3과목 데이터베이스 구축

41 릴레이션에 대한 설명으로 옳은 것은? 관계모델

- ① 튜플의 수를 차수(Degree)라 한다
- ② 속성의 수를 카디널리티(Cardinality)라 한다
- ③ 튜플의 수를 카디널리티, 속성의 수를 차수라 한다
- ④ 차수와 카디널리티는 항상 같은 값을 갖는다

정답 ③

정답 근거 카디널리티 = 튜플(행)의 수, 차수(Degree) = 속성(열)의 수입니다.

함정 ①②는 둘을 정확히 맞바꾼 보기입니다. 용어를 반대로 외우면 두 문제를 한 번에 틀립니다.

강사 한마디 저는 '카드(Card)는 한 장씩 쌓인다'로 기억시킵니다. 카드가 한 장 한 장 쌓이듯 행이 쌓이는 것이 카디널리티예요. 남은 하나가 자동으로 차수(열)가 됩니다. 참고로 릴레이션의 성질도 함께 나옵니다 — 튜플에 순서가 없고, 중복 튜플이 없으며, 모든 속성값은 원자값입니다.

42 키(Key)에 대한 설명으로 옳지 않은 것은? 키

- ① 후보키는 유일성과 최소성을 모두 만족한다
- ② 기본키는 NULL 값을 가질 수 없다
- ③ 슈퍼키는 유일성은 만족하지만 최소성은 만족하지 않을 수 있다
- ④ 외래키는 반드시 유일한 값을 가져야 한다

정답 ④

정답 근거 외래키(FK)는 다른 테이블의 기본키를 참조할 뿐, 중복될 수 있고 NULL도 가능합니다. 한 부서에 여러 사원이 속하듯 같은 부서번호가 여러 행에 나타납니다.

함정 '키'라는 이름 때문에 모든 키가 유일해야 한다고 착각합니다. 유일성이 필수인 것은 슈퍼키·후보키·기본키이고, 외래키는 성격이 다릅니다.

강사 한마디 외래키는 '키'가 아니라 '연결고리'라고 생각하는 편이 정확합니다. 자기 테이블에서 행을 구분하는 역할이 아니라, 다른 테이블을 가리키는 포인터니까요. 그래서 중복도 되고 NULL도 됩니다. 참조 무결성 문제와 세트로 나오니 'FK는 참조 대상에 실제 존재하는 값이거나 NULL'까지 함께 외우세요.

43 학생 테이블의 '학과코드'가 학과 테이블에 존재하지 않는 값을 갖지 않도록 보장하는 제약 조건은? 무결성

- ① 개체 무결성
- ② 도메인 무결성
- ③ 참조 무결성
- ④ 고유 무결성

정답 ③

정답 근거 외래키 값이 참조하는 테이블의 기본키에 실제로 존재해야 한다는 규칙이 참조 무결성입니다.

함정 ① 개체 무결성은 기본키가 NULL·중복이면 안 된다는 규칙으로, 테이블 내부 이야기입니다. 이 문제는 두 테이블 사이의 관계라 참조 무결성입니다.

강사 한마디 판단 기준은 간단합니다 — 테이블이 두 개 등장하면 참조 무결성, 한 테이블 안의 기본키 이야기면 개체 무결성, 값의 범위·타입 이야기면 도메인 무결성. 참조 무결성과 함께 나오는 옵션이 CASCADE(같이 삭제)·SET NULL·RESTRICT(삭제 거부)인데, 이것도 자주 물어봅니다.

44 다음 설명에 해당하는 정규형은? 정규화

기본키가 아닌 모든 속성이 기본키에 완전 함수 종속되어야 하며, 부분 함수 종속을 제거한 상태

- ① 제1정규형
- ② 제3정규형
- ③ 제2정규형
- ④ BCNF

정답 ③

정답 근거 부분 함수 종속 제거 = 제2정규형(2NF)입니다. 복합키의 일부에만 종속된 속성을 분리합니다.

함정 ② 3NF는 이행 함수 종속($A \rightarrow B \rightarrow C$) 제거이고, ④ BCNF는 모든 결정자가 후보키가 되도록 하는 단계입니다. 셋 다 '종속 제거'라는 표현을 써서 어느 종속인지 정확히 봐야 합니다.

강사 한마디 정규화는 제거하는 대상만 붙잡으면 됩니다 — 1NF는 반복(비원자값), 2NF는 부분, 3NF는 이행, BCNF는 결정자, 4NF는 다치, 5NF는 조인. 앞 글자로 '도·부·이·결·다·조'. 그리고 힌트 하나 — 부분 함수 종속은 복합키(키가 2개 이상)일 때만 생깁니다. 지문에 복합키가 보이면 2NF 이야기일 확률이 높습니다.

45 정규화되지 않은 테이블에서 발생할 수 있는 이상(Anomaly) 현상이 아닌 것은? 이상현상

- ① 삽입 이상
- ② 삭제 이상
- ③ 갱신 이상
- ④ 검색 이상

정답 ④

정답 근거 이상 현상은 삽입·삭제·갱신 세 가지입니다. 조회(검색)는 데이터를 바꾸지 않으므로 이상 현상이 생기지 않습니다.

함정 '검색 이상'은 존재하지 않는 용어인데, 나머지와 나란히 놓으면 자연스러워 보입니다. 없는 것을 만들어 넣는 전형적인 오답 설계입니다.

강사 한마디 세 가지가 모두 데이터를 변경하는 연산이라는 공통점을 기억하세요. 그래서 조회는 애초에 후보가 아닙니다. 각각의 정의도 한 줄로 — 삽입 이상은 원치 않는 값까지 넣어야 하는 것, 삭제 이상은 지우면 필요한 정보까지 사라지는 것, 갱신 이상은 중복된 값 일부만 바뀌어 불일치가 생기는 것입니다.

46 관계대수에서 릴레이션의 특정 속성(열)만 추출하는 연산과 그 기호로 옳은 것은? 관계대수

- ① Select, σ
- ② Join, $\bowtie$
- ③ Project, π
- ④ Division, $\div$

정답 ③

정답 근거 Project(π , 파이)가 원하는 열만 뽑는 수직적 연산입니다. SQL의 SELECT 열 목록에 해당합니다.

함정 ①Select(σ)는 이름이 SQL의 SELECT와 같아 착각하기 쉽지만, 관계대수의 Select는 조건에 맞는 행을 고르는 연산으로 SQL의 WHERE에 해당합니다.

강사 한마디 이 이름 충돌이 이 문제의 핵심 함정입니다. 정리하면 — 관계대수 σ (Select) = SQL의 WHERE(행), 관계대수 π (Project) = SQL의 SELECT절(열). 기호 모양으로도 외울 수 있습니다: π 는 기둥 두 개(세로=열). 저는 이렇게 가르치는데 실제로 잘 안 잊습니다.

47 SQL 명령어의 분류가 옳지 않은 것은? SQL

- ① DDL — CREATE, ALTER, DROP
- ② DML — SELECT, INSERT, UPDATE, DELETE
- ③ DCL — GRANT, REVOKE
- ④ **TCL — TRUNCATE, ALTER**

정답 ④

정답 근거 TCL(트랜잭션 제어)은 **COMMIT·ROLLBACK·SAVEPOINT**입니다. TRUNCATE와 ALTER는 구조를 다루는 **DDL**입니다.

함정 TRUNCATE가 '데이터를 지운다'는 점 때문에 DML이나 TCL로 오해되기 쉽습니다. 하지만 TRUNCATE는 **구조는 두고 전체 행을 즉시 비우는 DDL**이며 **롤백이 안 됩니다**.

강사 한마디 삭제 3형제를 확실히 정리하고 가세요. **DELETE**=DML, 조건 지정 가능, **롤백 가능**. **TRUNCATE**=DDL, 전체 삭제, 빠름, **롤백 불가**. **DROP**=DDL, **테이블 자체를 없앴**. 이 세 개의 차이는 거의 매 회 출제되고, 실기에서도 나옵니다.

48 다음 SQL문에서 실행 순서가 가장 마지막인 절은? SQL

```
SELECT 부서, COUNT(*)  
FROM 사원  
WHERE 급여 > 3000  
GROUP BY 부서  
HAVING COUNT(*) > 2  
ORDER BY 부서;
```

- ① WHERE
- ② GROUP BY
- ③ SELECT
- ④ **ORDER BY**

정답 ④

정답 근거 실행 순서는 **FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY**입니다. 정렬은 결과가 다 만들어진 뒤에 하므로 **ORDER BY가 마지막**입니다.

함정 SELECT가 맨 위에 쓰여 있어 먼저 실행된다고 착각하기 쉽습니다. **작성 순서와 실행 순서는 다릅니다**.

강사 한마디 이 순서를 알면 떨어 오는 문제가 여럿 풀립니다. 예를 들어 **SELECT에서 붙인 별칭(AS)을 WHERE에서 못 쓰는 이유**가 바로 WHERE이 SELECT보다 먼저 실행되기 때문이고, **ORDER BY에서는 별칭을 쓸 수 있는 이유**는 SELECT 다음이기 때문입니다. 순서 하나로 여러 문제가 설명됩니다.

49 집계 함수의 결과에 조건을 걸어 그룹을 걸러낼 때 사용하는 절은? SQL

- ① WHERE
- ② ORDER BY
- ③ GROUP BY
- ④ **HAVING**

정답 ④

정답 근거 **HAVING**은 GROUP BY로 묶인 **그룹**에 조건을 겁니다. `HAVING COUNT(*) ≥ 2` 처럼 집계 함수를 조건으로 쓸 수 있습니다.

함정 WHERE에 집계 함수를 쓰면 오류가 납니다. WHERE는 그룹화 **이전**에 개별 행을 거르는 단계라, 그 시점엔 COUNT 값이 존재하지 않기 때문입니다.

강사 한마디 '집계 함수가 조건에 들어가면 무조건 **HAVING**' 하나면 끝납니다. 반대로 개별 행의 값(급여, 날짜 등)으로 거르는 것은 WHERE이고요. 실무 팁도 하나 — 가능하면 WHERE로 먼저 줄이고 그다음 그룹화하는 것이 성능에 유리합니다. 이런 성능 관점을 묻는 문제도 가끔 나옵니다.

50 두 테이블에서 양쪽 모두에 매칭되는 행만 결과에 포함하는 조인은? SQL

- ① **INNER JOIN**
- ② LEFT OUTER JOIN
- ③ FULL OUTER JOIN
- ④ CROSS JOIN

정답 ①

정답 근거 **INNER JOIN**은 조인 조건을 만족하는 행, 즉 **교집합**만 결과로 냅니다.

함정 ②LEFT OUTER JOIN은 **왼쪽 테이블 전부**를 남기고 짝이 없으면 NULL을 채웁니다. '조인'이라는 말만 보고 고르면 헷갈립니다.

강사 한마디 조인은 **벤 다이어그램**으로 그리면 전부 정리됩니다. INNER=겹치는 부분, LEFT=왼쪽 원 전체, RIGHT=오른쪽 원 전체, FULL=두 원 전체, CROSS=모든 조합(**곱집합**, 행 수가 $m \times n$). 특히 CROSS JOIN은 **결과 행 수를 계산하라**는 문제로 나오니 곱셈이라는 것만 기억하세요.

51 다음 SQL문의 실행 결과로 옳은 것은? (사원 테이블에 급여가 NULL인 행이 2건 포함되어 있고, 전체 행은 10건이다) SQL

```
SELECT COUNT(*), COUNT(급여) FROM 사원;
```

- ① 10, 8
- ② 10, 10
- ③ 8, 8
- ④ 8, 10

정답 ①

정답 근거 COUNT(*)는 NULL 여부와 무관하게 모든 행을 세므로 10입니다. 반면 COUNT(열이름)은 그 열이 NULL이 아닌 행만 세므로 10 - 2 = 8입니다.

함정 둘을 같다고 생각하면 ②을 고릅니다. 집계 함수는 NULL을 무시한다는 성질이 이 문제의 전부입니다.

강사 한마디 NULL은 3과목에서 가장 자주 나오는 숨은 함정입니다. 기억할 규칙 세 가지 — (1) COUNT(*)만 NULL을 포함해서 셉니다. (2) SUM·AVG·MAX·MIN은 NULL을 무시합니다(그래서 AVG는 NULL을 0으로 치지 않습니다). (3) NULL 비교는 = NULL 이 아니라 IS NULL 로 해야 합니다. 이 세 개가 문제로 자주 나옵니다.

52 테이블의 모든 행을 삭제하되 테이블 구조는 유지하며, 롤백이 불가능한 명령은? SQL

- ① DELETE FROM 테이블
- ② TRUNCATE TABLE 테이블
- ③ DROP TABLE 테이블
- ④ ALTER TABLE 테이블

정답 ②

정답 근거 TRUNCATE는 전체 행을 한 번에 비우고 구조는 남깁니다. DDL이라 자동 커밋되어 롤백할 수 없습니다.

함정 ①DELETE도 전체 삭제가 가능해 헛갈리지만, DELETE는 DML이라 롤백이 됩니다. 문제의 '롤백 불가'가 결정적 단서입니다.

강사 한마디 문제에 조건이 두 개 이상 붙으면 그 조합으로 답이 하나로 좁혀집니다. 여기서는 '구조 유지'로 DROP이 빠지고, '롤백 불가'로 DELETE가 빠집니다. 이렇게 조건을 하나씩 소거하는 습관을 들이면 애매한 문제도 안전하게 풀립니다.

53 뷰(View)에 대한 설명으로 옳지 않은 것은? 뷰

- ① 하나 이상의 기본 테이블로부터 유도된 가상 테이블이다
- ② 필요한 열만 보여줄 수 있어 보안에 유리하다
- ③ 뷰를 통해 데이터를 조회하면 실제 테이블의 데이터를 참조한다
- ④ 뷰는 독립적인 물리적 저장 공간을 가지므로 데이터가 중복 저장된다

정답 ④

정답 근거 뷰는 정의(SELECT문)만 저장되는 가상 테이블입니다. 별도의 물리적 데이터 저장 공간을 갖지 않으므로 중복 저장이 없습니다.

함정 '테이블'이라는 이름 때문에 실제 데이터를 갖고 있다고 착각하기 쉽습니다. '가상'이라는 수식어가 핵심입니다.

강사 한마디 뷰는 '창문'이라고 생각하세요. 창문 너머 풍경(데이터)은 원래 테이블에 있고, 창문은 어디를 어떻게 볼지만 정해 놓은 것입니다. 그래서 원본이 바뀌면 뷰의 결과도 바뀝니다. 함께 나오는 특징 — 뷰는 독립적인 인덱스를 만들 수 없고, 정의를 변경할 수 없어 바꾸려면 DROP 후 재생성해야 합니다.

54 인덱스(Index)에 대한 설명으로 옳은 것은? 인덱스

- ① 인덱스를 많이 만들수록 삽입·수정 성능이 향상된다
- ② 인덱스는 검색 속도를 높이지만 삽입·삭제 시 추가적인 갱신 비용이 발생한다
- ③ 인덱스는 테이블당 하나만 생성할 수 있다
- ④ 인덱스를 생성하면 원본 테이블의 데이터가 정렬되어 저장된다

정답 ②

정답 근거 인덱스는 조회를 빠르게 하는 대신, 데이터가 바뀔 때마다 인덱스도 함께 갱신해야 해서 삽입·수정·삭제에는 부담이 됩니다.

함정 ①은 정확히 반대로 서술했습니다. 인덱스가 많을수록 쓰기 성능은 나빠집니다. ③도 틀립니다 — 인덱스는 여러 개 만들 수 있습니다.

강사 한마디 책의 색인으로 이해하면 전부 설명됩니다. 색인이 있으면 원하는 쪽을 빨리 찾지만(조회 ↑), 책 내용을 고칠 때마다 색인도 다시 만들어야 하죠(쓰기 ↓). 그래서 실무에서는 조회가 잦은 열에만 인덱스를 겁니다. '많을수록 좋다'는 서술이 보이면 그건 오답입니다.

55 트랜잭션의 특성 중 다음 설명에 해당하는 것은? 트랜잭션

트랜잭션이 성공적으로 완료되면 그 결과는 시스템에 장애가 발생하더라도 영구적으로 반영되어야 한다

- ① 원자성(Atomicity)
- ② 일관성(Consistency)
- ③ 격리성(Isolation)
- ④ **지속성(Durability)**

정답 ④

정답 근거 완료된 결과가 **영구히 유지**되어야 한다는 것이 **지속성(Durability)**입니다. 이를 위해 로그와 체크포인트를 사용합니다.

함정 ①원자성은 **전부 반영되거나 전혀 반영되지 않아야** 한다는 것으로, '완료 후'가 아니라 '완료 여부'에 관한 성질입니다.

강사 한마디 ACID는 시점으로 나누면 명쾌합니다 — **원자성은 실행 중**(다 되거나 다 안 되거나), **격리성도 실행 중**(다른 트랜잭션과 간섭 없이), **지속성은 완료 후**(영구 보존), **일관성은 전후 비교**(제약 조건 유지). 지문에 '장애가 나도·영구히'가 있으면 지속성입니다.

56 두 개 이상의 트랜잭션이 서로 상대가 점유한 자원을 기다리며 무한정 대기하는 상태는? 병행제어

- ① **교착상태(Deadlock)**
- ② 기아상태(Starvation)
- ③ 경쟁상태(Race Condition)
- ④ 로킹(Locking)

정답 ①

정답 근거 서로가 서로를 기다려 **아무도 진행하지 못하는** 상태가 **교착상태**입니다. 발생 조건은 상호배제·점유와 대기·비선점·환형 대기 **네 가지가 모두** 성립할 때입니다.

함정 ②기아상태는 **우선순위에 계속 밀려** 자원을 못 받는 것으로, 서로 기다리는 것이 아니라 **한쪽만 계속 밀리는** 상황입니다.

강사 한마디 둘의 차이를 그림으로 잡으세요 — **교착**은 서로 마주보고 **멈춤(양방향)**, **기아**는 혼자 계속 뒤로 **밀림(일방향)**. 그리고 교착 4조건은 '**하나만 깨도 예방된다**'는 점이 자주 출제됩니다. 특히 '**비선점**'이 조건이라는 것 — '선점'이라고 적힌 보기는 틀린 것이니 주의하세요.

57 데이터베이스 장애 발생 시, 이미 커밋된 트랜잭션의 변경 내용을 로그를 이용해 다시 반영하는 회복 기법은? 회복

- ① UNDO
- ② **REDO**
- ③ ROLLBACK
- ④ CHECKPOINT

정답 ②

정답 근거 REDO는 커밋은 되었으나 디스크에 반영되지 못한 변경을 로그를 보고 다시 수행하는 것입니다. 반대로 UNDO는 완료되지 못한 트랜잭션의 변경을 취소합니다.

함정 ③ROLLBACK은 사용자가 명시적으로 트랜잭션을 취소하는 명령어(TCL)이지 회복 기법의 명칭이 아닙니다. 의미가 UNDO와 비슷해 헷갈립니다.

강사 한마디 방향으로 외우세요 — REDO는 앞으로(다시 하기), UNDO는 뒤로(되돌리기). 판단 기준은 커밋 여부입니다. 커밋된 것은 REDO, 커밋 안 된 것은 UNDO. 이 한 줄이면 어떤 상황을 줘도 분류됩니다. CHECKPOINT는 회복 시작 지점을 표시해 회복 범위를 줄이는 장치입니다.

58 데이터베이스 3단계 스키마 중, 조직 전체의 관점에서 데이터베이스 전체의 논리적 구조를 정의하는 것은? 스키마

- ① 외부 스키마
- ② **개념 스키마**
- ③ 내부 스키마
- ④ 물리 스키마

정답 ②

정답 근거 개념 스키마는 조직 전체를 아우르는 통합된 논리 구조로, 데이터베이스당 하나만 존재하며 DBA가 관리합니다.

함정 ①외부 스키마는 사용자·응용별 관점이라 여러 개가 존재합니다. '전체'와 '개별'을 구분하는 것이 핵심입니다.

강사 한마디 개수로 외우면 편합니다 — 외부는 여러 개(사용자마다), 개념은 하나, 내부도 하나. 그리고 이 3단계를 나누는 이유가 데이터 독립성인데, 이것도 세트로 출제됩니다. 논리적 독립성 = 개념↔외부(논리 구조가 바뀌어도 응용은 영향 최소), 물리적 독립성 = 내부↔개념(저장 방식이 바뀌어도 논리 구조는 그대로).

59 성능 향상을 위해 정규화된 테이블을 의도적으로 통합하거나 중복을 허용하는 것은? 설계

- ① 정규화
- ② 반정규화(De-normalization)
- ③ 무결성 제약
- ④ 트랜잭션 분리

정답 ②

정답 근거 반정규화는 조인을 줄여 조회 성능을 높이려고 일부러 중복을 허용하는 설계 기법입니다. 정규화의 반대 방향 절충입니다.

함정 '정규화'와 이름이 비슷해 지문을 대충 읽으면 ①을 고릅니다. 의도적으로 중복을 허용한다는 부분이 결정적입니다.

강사 한마디 정규화와 반정규화는 **맞바꿈(trade-off)** 관계입니다. 정규화하면 **중복 ↓ 무결성 ↑**이지만 조인이 늘어 **조회 성능 ↓**. 반정규화는 그 반대고요. 그래서 시험에서 '반정규화의 단점'을 물으면 답은 **데이터 중복으로 인한 무결성 저하·갱신 비용 증가**입니다. 장단점을 짚으로 정리해 두세요.

60 E-R 다이어그램의 표기법으로 옳지 않은 것은? ERD

- ① 개체(Entity)는 사각형으로 표현한다
- ② 속성(Attribute)은 타원으로 표현한다
- ③ 관계(Relationship)는 마름모로 표현한다
- ④ 기본키 속성은 이중 사각형으로 표현한다

정답 ④

정답 근거 기본키 속성은 타원 안의 이름에 **밑줄**을 그어 표시합니다. 이중 사각형은 **약한 개체**를 나타내는 기호입니다.

함정 ①②③이 모두 맞아서 앞에서 답을 고르기 쉽습니다. 기본키 표기까지 정확히 알아야 걸러지는 문제입니다.

강사 한마디 ERD 기호는 **사각형(개체) · 타원(속성) · 마름모(관계) · 선(연결) · 밑줄(기본키)** 다섯 개면 충분합니다. 여기에 **이중 사각형=약한 개체, 이중 타원=다중값 속성, 점선 타원=유도 속성**까지 알면 어떤 변형도 대응됩니다. 이 문제처럼 **맞는 것 세 개 + 미묘하게 틀린 하나** 구조가 3과목의 전형입니다.

4과목 프로그래밍 언어 활용

61 다음 C 프로그램의 출력 결과는? C 언어

```
int main()
{
    int a = 5, b, c;
    b = a++;
    c = ++a;
    printf("%d %d %d", a, b, c);
    return 0;
}
```

① 7 5 7

② 6 5 6

③ 7 6 7

④ 5 5 7

정답 ①

정답 근거 `b = a++` 는 **b에 현재값 5를 먼저 넣고** a를 6으로 올립니다(b=5, a=6). 이어서 `c = ++a` 는 **a를 7로 먼저 올린 뒤** c에 대입합니다(a=7, c=7). 따라서 **7 5 7**.

참정 연산이 두 줄이라 a의 최종값을 6으로 착각하기 쉽습니다. a는 **두 번 증가**했습니다.

강사 한마디 증감 연산자는 ++의 위치가 곧 순서입니다. 뒤에 붙으면(a++) '쓰고 올린다', 앞에 붙으면(++a) '올리고 쓴다'. 이 문제처럼 여러 줄이 이어지면 **변수 값을 줄마다 옆에 적으면서** 따라가세요. 머리로만 하면 마지막 값에서 꼭 한 번 틀립니다. 4과목 코드 문제는 손으로 적는 사람이 이깁니다.

62 다음 C 코드의 출력 결과는? C 언어

```
int a[5] = {1, 2, 3, 4, 5};
int *p = a;
p++;
printf("%d", *p + *(p + 2));
```

- ① 4
- ② 8
- ③ 6
- ④ 5

정답 ③

정답 근거 `p = a` 이면 `p`는 `a[0]`을 가리킵니다. `p++` 로 `a[1]`로 이동하므로 `*p` = 2입니다. `*(p+2)` 는 `a[3]` = 4이므로 합은 6입니다.

함정 `p++` 를 놓치고 `*p` 를 1로 계산하면 `1+3=4`(①)가 됩니다. 포인터 이동을 반드시 반영해야 합니다.

강사 한마디 포인터 문제는 '지금 몇 번 칸을 가리키는가'만 추적하면 됩니다. 배열 위에 인덱스를 적고 화살표를 옮겨 보세요. 핵심 등식은 `*(p+i) = p[i]` 입니다. 이 둘이 완전히 같다는 걸 알면 포인터 표기가 나와도 그냥 배열 문제로 바꿔 풀 수 있습니다.

63 다음 C 코드의 실행 결과 sum의 값은? C 언어

```
int i = 0, sum = 0;
while (i < 5)
{
    i++;
    if (i % 2 == 1) continue;
    sum += i;
}
```

- ① 6
- ② 9
- ③ 15
- ④ 4

정답 ①

정답 근거 `continue` 는 홀수일 때 남은 부분을 건너뛰고 다음 반복으로 갑니다. 결국 짝수만 더해지므로 `2 + 4 = 6`입니다.

함정 ③15는 1~5를 모두 더한 값으로, `continue`를 무시했을 때 나옵니다. ②9는 홀수(1+3+5)를 더한 경우입니다.

강사 한마디 `continue`와 `break`를 정확히 구분하세요 — `continue`는 이번 회차만 건너뛰고 반복은 계속, `break`는 반복문 자체를 탈출합니다. 이 문제에서 `continue`를 `break`로 바꾸면 `sum`은 0이 됩니다(`i=1`에서 바로 탈출). 출제자는 이 둘을 바꿔 내는 걸 좋아하니, 코드에서 두 단어를 보면 표시부터 하세요.

64 C에서 `int x = 12;` 일 때 `x >> 2` 의 결과는? C 언어

- ① 3
- ② 6
- ③ 24
- ④ 48

정답 ①

정답 근거 오른쪽 시프트 `>>n` 은 **2ⁿ으로 나누기**입니다. $12 \div 2^2 = 12 \div 4 = 3$ 입니다. (2진수 1100 → 0011)

함정 방향을 반대로 알아 곱하기로 계산하면 48(④)이 나옵니다. **왼쪽(<<)이 곱하기, 오른쪽(>>)이 나누기**입니다.

강사 한마디 화살표가 가리키는 쪽으로 **비트가 밀려간다고** 생각하면 방향이 안 헛갈립니다. 왼쪽으로 밀면 자릿수가 커지니 $\times 2$, 오른쪽으로 밀면 작아지니 $\div 2$. 비트 연산은 이것 말고 **XOR(^)**도 잘 나오는데, **같으면 0, 다르면 1**이고 **같은 수끼리 XOR하면 0**이라는 성질을 기억하세요.

65 다음 Java 코드의 출력 결과는? Java

```
class Parent {  
    void print() { System.out.print("Parent"); }  
}  
class Child extends Parent {  
    void print() { System.out.print("Child"); }  
}  
  
// main  
Parent p = new Child();  
p.print();
```

- ① Parent
- ② ParentChild
- ③ Child
- ④ 컴파일 오류

정답 ③

정답 근거 참조 변수의 타입은 Parent지만 **실제 생성된 객체는 Child**입니다. 오버라이딩된 메서드는 **실행 시점의 실제 객체**를 기준으로 호출되므로(동적 바인딩) **Child**가 출력됩니다.

함정 선언 타입(Parent)을 보고 ①을 고르기 쉽습니다. 하지만 자바에서 **메서드는 객체 기준**으로 결정됩니다.

강사 한마디 이게 **다형성**의 핵심이고 필기에서 매우 자주 나옵니다. 한 문장으로 '**변수는 껍데기, 실행은 알맹이(실제 객체)**'라고 기억하세요. 단 예외가 있습니다 — **필드(변수)와 static 메서드는 선언 타입 기준**으로 결정됩니다. 이 예외를 이용한 함정 문제도 나오니 '메서드만 객체 기준'이라고 정확히 새기세요.

66 Java에 대한 설명으로 옳지 않은 것은? Java

- ① static 메서드는 오버라이딩할 수 없다
- ② 생성자는 상속되지 않는다
- ③ 클래스의 다중 상속을 지원한다
- ④ final로 선언된 메서드는 오버라이딩할 수 없다

정답 ③

정답 근거 Java는 클래스의 다중 상속을 지원하지 않습니다. 여러 부모에서 같은 메서드를 물려받을 때 생기는 모호함(다이아몬드 문제) 때문이며, 대신 인터페이스는 여러 개 구현할 수 있습니다.

함정 C++가 다중 상속을 지원하기 때문에 자바도 된다고 착각하기 쉽습니다. 언어별로 다르다는 점이 함정입니다.

강사 한마디 '오버라이딩이 안 되는 것' 3종 세트를 묶어 외우세요 — **static · final · private** 메서드입니다. 여기에 **생성자는 상속 자체가 안 된다**까지 자주 나오는 지문입니다. 반대로 **인터페이스는 다중 구현 가능**이라는 것도 짝으로 기억하면, 이 유형은 어떻게 꼬아 내도 잡힙니다.

67 Java의 접근 제어자 중 같은 패키지 내부와 상속받은 자식 클래스에서 접근을 허용하는 것은? Java

- ① public
- ② default
- ③ protected
- ④ private

정답 ③

정답 근거 **protected**는 같은 패키지 + 다른 패키지의 자식 클래스까지 접근을 허용합니다. default(package-private)는 같은 패키지지만 허용하고 상속은 고려하지 않습니다.

함정 ②default와 헷갈립니다. 결정적 차이는 '상속받은 자식'인데, default는 다른 패키지의 자식에게 열리지 않습니다.

강사 한마디 범위를 넓은 순으로 **public > protected > default > private**으로 외우고, 각 경계를 한 단어로 — public=**전부**, protected=**패키지+자식**, default=**패키지**, private=**자기 클래스**. 문제에 '**자식·상속**'이 등장하면 거의 protected입니다. 캡슐화 개념 문제와 함께 나오면 '필드는 private, 접근은 메서드로'가 정답 방향입니다.

68 다음 Python 코드의 출력 결과는? Python

```
a = [1, 2, 3, 4, 5]
print(a[1:4])
```

- ① [1, 2, 3]
- ② [2, 3, 4, 5]
- ③ **[2, 3, 4]**
- ④ [1, 2, 3, 4]

정답 ③

정답 근거 슬라이싱 `a[시작:끝]` 은 시작은 포함, 끝은 제외합니다. 인덱스 1, 2, 3에 해당하는 **[2, 3, 4]**입니다.

함정 끝 인덱스를 포함한다고 생각하면 ②[2,3,4,5]가 됩니다. 끝은 **항상 미포함**이라는 규칙을 기계적으로 적용하세요.

강사 한마디 슬라이싱은 '**개수 = 끝 - 시작**'으로 계산하면 실수를 잡습니다. 여기선 $4-1=3$ 개니까 원소 세 개짜리 보기만 후보가 됩니다. 그것만으로 ②④가 걸러지죠. 함께 알아둘 관용구 — `a[::-1]` 은 역순, `a[:n]` 은 앞에서 n개, `a[-n:]` 은 뒤에서 n개입니다.

69 다음 Python 코드의 실행 결과로 출력되는 값은? Python

```
d = {'a': 1, 'b': 2}
d['a'] = 9
d['c'] = 3
print(len(d))
```

- ① 2
- ② 오류 발생
- ③ 4
- ④ **3**

정답 ④

정답 근거 딕셔너리는 키가 중복될 수 없습니다. `d['a']=9` 는 새 항목이 아니라 기존 'a'의 값을 덮어쓴 것이고, `d['c']=3` 만 새로 추가됩니다. 따라서 항목은 a, b, c **3개**입니다.

함정 대입을 모두 '추가'로 세면 4개(③)가 됩니다. 키가 이미 있으면 수정, 없으면 추가라는 동작이 핵심입니다.

강사 한마디 파이썬 자료형은 '**중복 허용 여부**'와 '**수정 가능 여부**' 두 축으로 정리하면 어떤 문제든 대응됩니다. **list**=순서O·중복O·수정O, **tuple**=순서O·중복O·수정X, **set**=순서X·중복X, **dict**=키 중복X·값은 자유. 이 표가 4과목 파이썬 문제의 절반을 커버합니다.

70 다음 Python 코드의 출력 결과는? Python

```
result = [x * 2 for x in range(1, 4)]  
print(result)
```

- ① [2, 4, 6]
- ② [2, 4, 6, 8]
- ③ [1, 2, 3]
- ④ [0, 2, 4]

정답 ①

정답 근거 range(1, 4) 는 1, 2, 3을 만들고(끝값 4 미포함), 각각 2를 곱해 [2, 4, 6]이 됩니다.

함정 range의 끝값을 포함한다고 보면 4까지 계산해 ②가 됩니다. 슬라이싱과 마찬가지로 **range도 끝값 미포함**입니다.

강사 한마디 파이썬에서 '끝은 포함하지 않는다'는 규칙은 range·슬라이싱에 공통으로 적용됩니다. 이거 하나로 두 유형이 동시에 정리돼요. range(a, b) 의 개수는 $b - a$ 라는 것도 계산에 유용합니다(여기선 $4 - 1 = 3$ 개). 리스트 컴프리헨션 자체는 어렵지 않으니 **range 범위**에만 집중하세요.

71 CPU 스케줄링 기법 중 선점(Preemptive) 방식에 해당하는 것은? 운영체제

- ① FCFS
- ② SJF
- ③ HRN
- ④ Round Robin

정답 ④

정답 근거 라운드 로빈(RR)은 시간 할당량(Time Quantum)이 끝나면 실행 중인 프로세스를 **강제로 중단**하고 다음으로 넘깁니다. 이것이 선점입니다.

함정 ①②③은 모두 비선점입니다. 특히 **SJF는 비선점, SRT는 선점**이라는 짝을 정확히 알아야 합니다(SRT가 SJF의 선점 버전).

강사 한마디 선점 = 뺏을 수 있다입니다. 선점형은 **RR·SRT·다단계 큐·선점 우선순위**, 비선점형은 **FCFS·SJF·HRN·기한부**. 구분 요령 — 시간을 쪼개거나 남은 시간을 계속 비교하면 선점, 한 번 잡으면 끝까지 하면 비선점입니다. RR은 응답 시간이 좋아 **시분할 시스템**에 쓰인다는 점도 함께 나옵니다.

72 HRN 스케줄링에서 대기 시간이 20, 서비스 시간이 5인 작업의 우선순위 값은? 운영 체제

- ① 4
- ② 25
- ③ 3
- ④ 5

정답 ④

정답 근거 HRN 우선순위 = (대기 시간 + 서비스 시간) ÷ 서비스 시간 = (20 + 5) ÷ 5 = 25 ÷ 5 = 5입니다. 값이 클수록 먼저 실행됩니다.

함정 분모에 서비스 시간을 넣는 것을 잊고 $20 \div 5 = 4$ (①)로 계산하는 실수가 가장 흔합니다. 분자에도 서비스 시간이 더해집니다.

강사 한마디 공식을 못 외웠어도 의미로 복원할 수 있습니다. HRN은 SJF의 단점인 긴 작업의 기아를 막으려고 기다린 시간에 가산점을 주는 방식입니다. 그래서 분자에 대기 시간이 들어가고, 짧은 작업이 유리하도록 서비스 시간으로 나눕니다. 계산 결과는 항상 1보다 큼니다(대기가 0이어도 1). 1 미만이 나오면 계산이 틀린 겁니다.

73 페이지 참조열이 다음과 같고 프레임이 3개일 때, LRU 알고리즘의 페이지 부재(Page Fault) 횟수는? 운영 체제

참조열 : 2 3 2 1 5 2 4

- ① 5회
- ② 4회
- ③ 6회
- ④ 7회

정답 ①

정답 근거 2(부재) 3(부재) 2(적중) 1(부재) → [2,3,1]. 5(부재, 가장 오래 안 쓴 3 교체) → [2,1,5]. 2(적중). 4(부재, 가장 오래 안 쓴 1 교체) → 총 5회.

함정 FIFO로 계산하면 6회가 나와 ③을 고르게 됩니다. LRU는 들어온 순서가 아니라 마지막으로 사용한 시점을 봅니다.

강사 한마디 LRU 계산은 표를 그려 프레임 상태를 매 단계 적는 것이 정석입니다. 그리고 반드시 적중(hit)한 페이지도 '사용 시'각을 갱신해야 합니다 — 이 갱신을 빠뜨려서 FIFO와 같은 답을 내는 실수가 가장 많습니다. 위에서 2가 중간에 적중하면서 살아남은 것이 그 예입니다. 표 그리기는 30초를 아끼려다 틀리는 대표적 문제입니다.

74 UNIX/Linux에서 파일 권한이 `rwxr-xr--` 일 때, 이를 8진수로 나타낸 것은? 운영체제

- ① 744
- ② 754
- ③ 755
- ④ 764

정답 ②

정답 근거 `r`=4, `w`=2, `x`=1로 계산합니다. 소유자 `rw``x` = 4+2+1=7, 그룹 `r-x` = 4+0+1=5, 기타 `r--` = 4+0+0=4 → 754.

함정 ③ 755는 기타에도 실행 권한(x)이 있는 경우입니다. 마지막 세 칸이 `r--` 인지 `r-x` 인지 꼼꼼히 봐야 합니다.

강사 한마디 권한 문자열은 세 칸씩 끊어서 읽는 습관을 들이세요 — `rw``x` | `r-x` | `r--`. 그리고 자주 쓰는 조합은 값을 통째로 외워두면 빠릅니다: 777(전부 허용), 755(소유자만 쓰기), 644(실행 불가 일반 파일), 700(소유자 전용). 반대로 숫자를 문자로 바꾸는 문제도 나오니 양방향으로 연습하세요.

75 OSI 7계층 중 라우터가 동작하며, 논리 주소(IP)를 이용한 경로 설정을 담당하는 계층은? 네트워크

- ① 데이터링크 계층
- ② 네트워크 계층
- ③ 전송 계층
- ④ 세션 계층

정답 ②

정답 근거 네트워크 계층(3계층)이 IP 주소를 기반으로 경로를 결정(라우팅)합니다. 대표 장비가 라우터, 대표 프로토콜이 IP·ICMP입니다.

함정 ① 데이터링크는 MAC 주소를 쓰는 같은 네트워크 안의 전달을 담당합니다(장비: 스위치·브리지). '주소'라는 말만 보고 고르면 헷갈립니다.

강사 한마디 계층별 주소와 장비를 세트로 외우면 관련 문제가 한꺼번에 풀립니다 — 1계층: 비트, 허브·리피터 / 2계층: MAC 프레임, 스위치·브리지 / 3계층: IP 패킷, 라우터 / 4계층: 포트·세그먼트, TCP/UDP. 순서는 '물·데·네·전·세·표·응'. 이 표 하나로 4과목 네트워크 문제 서너 개를 확보합니다.

76 TCP와 UDP에 대한 설명으로 옳은 것은? 네트워크

- ① UDP는 3-way handshake로 연결을 설정한다
- ② **TCP는 데이터의 순서 보장과 재전송을 지원한다**
- ③ TCP는 UDP보다 전송 속도가 빠르다
- ④ UDP는 흐름 제어와 혼잡 제어를 모두 제공한다

정답 ②

정답 근거 TCP는 연결형 프로토콜로 **순서 보장·오류 검출·재전송·흐름 제어**를 제공합니다. 그래서 신뢰성이 높습니다.

함정 ①의 3-way handshake는 **TCP**의 연결 설정 과정입니다. ③도 반대로, 신뢰성 처리 때문에 **TCP가 더 느립니다**.

감사 한마디 둘의 성격은 **맞바꿈 관계**로 이해하세요 – **TCP는 확실하지만 느리고, UDP는 빠르지만 확실하지 않습니다**. 그래서 용도가 갈립니다: 파일·웹·메일은 TCP, **실시간 스트리밍·화상통화·DNS**는 UDP. '조금 끊겨도 빨라야 하는 것'은 UDP라고 기억하면 용도 문제도 함께 풀립니다.

77 IP 주소 192.168.10.0/26 으로 서브네팅했을 때, 서브넷 하나당 사용 가능한 호스트 수는? 네트워크

- ① 30개
- ② **62개**
- ③ 126개
- ④ 64개

정답 ②

정답 근거 /26이면 호스트 비트가 $32 - 26 = 6$ 비트입니다. $2^6 = 64$ 개 주소 중 **네트워크 주소와 브로드캐스트 주소 2개를 빼면** $64 - 2 = 62$ 개입니다.

함정 ④ 64는 **2를 빼지 않은** 값입니다. 이 '-2'를 잊는 것이 서브넷 문제의 최대 실수 요인입니다.

감사 한마디 공식은 $2^{(32-\text{프리픽스})} - 2$ 하나뿐입니다. 자주 나오는 값은 아예 외워두세요 – /24 → 254, /25 → 126, /26 → 62, /27 → 30, /28 → 14. 규칙이 보이시죠? 프리픽스가 1 늘 때마다 대략 **절반**이 됩니다. 이 다섯 개만 알면 서브넷 문제는 계산 없이 즉답할 수 있습니다.

78 IP 주소를 물리 주소(MAC)로 변환해 주는 프로토콜은? 네트워크

- ① DNS
- ② DHCP
- ③ **ARP**
- ④ ICMP

정답 ③

정답 근거 ARP(Address Resolution Protocol)는 통신 상대의 **IP 주소로부터 MAC 주소를 알아냅니다**. 반대로 MAC에서 IP를 알아내는 것은 RARP입니다.

함정 ①DNS는 **도메인 이름 → IP** 변환이라 '변환'이라는 공통점 때문에 헷갈립니다. 무엇을 무엇으로 바꾸는지가 갈림길입니다.

강사 한마디 이 넷은 '무엇을 하는가' 한 줄씩만 잡으면 됩니다 — **ARP: IP→MAC, DNS: 도메인→IP, DHCP: IP 자동 할당, ICMP: 오류 보고·ping**. 실생활로 연결하면 더 쉽습니다. 인터넷이 안 될 때 ping 을 치죠? 그게 ICMP고, 공유기가 IP를 자동으로 주는 게 DHCP입니다.

79 객체지향의 특징 중, 데이터와 그 데이터를 처리하는 함수를 하나로 묶고 내부 구현을 감추는 것은? 프로그래밍

- ① 상속
- ② **캡슐화**
- ③ 다형성
- ④ 추상화

정답 ②

정답 근거 **캡슐화**는 데이터(속성)와 메서드를 한 클래스로 묶고, 외부에서 내부에 직접 접근하지 못하게 **정보를 은닉**합니다. private 필드 + public 메서드가 전형적인 구현입니다.

함정 ④추상화와 혼동됩니다. 추상화는 **공통 특성만 뽑아 단순화**하는 개념이고, 캡슐화는 **묶고 감추는** 것입니다.

강사 한마디 네 가지를 한 단어로 못 박아 두세요 — **캡슐화=묶고 감춤, 상속=물려받음, 다형성=같은 이름 다른 동작, 추상화=핵심만 남김**. 캡슐화의 **효과**를 묻는 문제도 나오는데, 답은 **결합도가 낮아지고 재사용성·유지보수성이 높아진다**입니다. 1과목 결합도 개념과 연결되는 지점입니다.

80 프로그래밍 언어의 분류가 옳지 않은 것은? 프로그래밍

- ① C — 절차적 언어
- ② Java — 객체지향 언어
- ③ Python — 스크립트 언어
- ④ LISP — 절차적 언어

정답 ④

정답 근거 LISP는 대표적인 **함수형 언어**입니다. 함수형 언어에는 LISP 외에 Haskell, Scheme, Erlang 등이 있습니다.

함정 LISP가 오래된 언어라 '옛날 = 절차적'이라고 넘겨짚기 쉽습니다. 오래됐지만 계열은 함수형입니다.

강사 한마디 언어 분류는 **대표 선수 한 명씩만** 기억하면 됩니다 — **절차적: C, FORTRAN / 객체지향: Java, C++, C# / 스크립트: Python, JavaScript / 함수형: LISP, Haskell**. 시험에서 함수형의 예로 나오는 건 사실상 **LISP**뿐이니, 'LISP=함수형' 하나만 확실히 해도 이 유형은 방어됩니다.

5과목 정보시스템 구축관리

81 소프트웨어 프로세스의 성숙도를 초기·관리·정의·정량적 관리·최적화의 5단계로 평가하는 모델은? 방법론

- ① CMMI
- ② SPICE
- ③ COCOMO
- ④ PMBOK

정답 ①

정답 근거 CMMI(Capability Maturity Model Integration)가 조직의 프로세스 성숙도를 **5단계**로 평가합니다.

함정 ②SPICE(ISO/IEC 15504)도 프로세스 평가 모델이지만 **0~5의 6단계**로 나눕니다. 단계 수가 결정적 차이입니다.

강사 한마디 둘을 **숫자로** 구분하세요 — **CMMI는 5단계, SPICE는 6단계(0단계부터)**. 그리고 CMMI의 5단계 이름은 '**초기에 관리하고 정의해서 정량적으로 최적화한다**'라는 문장으로 외우면 순서까지 나옵니다. ③COCOMO는 비용 산정, ④PMBOK은 프로젝트 관리 지식체계로 성격이 아예 다릅니다.

82 LOC 기법으로 예측할 때 낙관치가 12,000, 기대치가 24,000, 비관치가 36,000 라인이라면 예측 LOC는? 비용산정

정

- ① 20,000
- ② 26,000
- ③ **24,000**
- ④ 30,000

정답 ③

정답 근거 예측치 = (낙관치 + 4 × 기대치 + 비관치) ÷ 6 = (12,000 + 96,000 + 36,000) ÷ 6 = 144,000 ÷ 6 = **24,000**입니다.

함정 기대치에 곱하는 4를 빠뜨리고 세 값의 단순 평균(=24,000)을 내면 **우연히 같은 답**이 나옵니다. 하지만 값이 대칭이 아닐 때는 달라지므로 공식을 정확히 써야 합니다.

감사 한마디 이 문제는 제가 일부러 대칭 수치를 넣었는데, 실제 시험에서는 **비대칭**으로 내서 단순 평균과 답이 갈리게 합니다. 그러니 반드시 **가중치 4**를 쓰세요. 이 공식은 PERT의 기대시간 공식과 **완전히 같습니다** — 일정 산정 문제에서도 그대로 쓰이니 한 번 외워 두 곳에서 씁니다.

83 COCOMO 모델의 개발 유형 중, 5만 라인 이하의 소규모이며 사무 처리·업무용 프로그램에 적합한 것은? 비용산정

- ① **Organic(조직형)**
- ② Semi-detached(반분리형)
- ③ Embedded(내장형)
- ④ Distributed(분산형)

정답 ①

정답 근거 **조직형(Organic)**은 5만 라인 이하 소규모 프로젝트에 해당합니다. 반분리형은 30만 라인 이하 중간 규모, 내장형은 **초대형·실시간·복잡한** 시스템입니다.

함정 ④ 분산형은 **COCOMO의 유형이 아닙니다**. 그럴듯한 이름을 만들어 넣은 오답입니다.

감사 한마디 세 유형은 **규모 숫자**로 외우는 게 가장 확실합니다 — **5만 이하 조직형, 30만 이하 반분리형, 30만 초과 내장형**. 그리고 성격으로도 연결됩니다: 내장형은 **하드웨어에 내장되는 실시간 제어**(항공·군사)라 가장 어렵고 비용 계수도 큼니다. '실시간·초대형'이라는 단어가 보이면 내장형입니다.

84 PERT/CPM에서 프로젝트의 전체 완료 기간을 결정하는, 가장 시간이 오래 걸리는 경로는? 프로젝트관리

- ① 최단 경로
- ② 병렬 경로
- ③ 임계 경로(Critical Path)
- ④ 여유 경로

정답 ③

정답 근거 임계 경로는 시작부터 종료까지의 경로 중 **소요 시간이 가장 긴** 경로이며, 이 경로의 길이가 곧 프로젝트 최소 완료 기간입니다.

함정 '가장 오래 걸리는 것이 완료 기간'이라는 점이 직관과 어긋나 헷갈립니다. 하지만 모든 작업이 끝나야 프로젝트가 끝나므로, **가장 늦게 끝나는 경로**가 기준이 됩니다.

강사 한마디 임계 경로의 핵심 성질은 '**여유 시간(Slack)이 0**'이라는 것입니다. 즉 **이 경로의 작업이 하루 지연되면 프로젝트 전체가 하루 지연됩니다**. 그래서 관리자는 여기에 자원을 집중하죠. 시험에서는 '지연되면 전체가 지연되는 경로'라는 서술로도 나오니 같은 말임을 알아두세요.

85 정보보안의 3대 요소(CIA)에 해당하지 않는 것은? 보안

- ① 기밀성(Confidentiality)
- ② 무결성(Integrity)
- ③ 가용성(Availability)
- ④ 확장성(Scalability)

정답 ④

정답 근거 보안 3대 요소는 **기밀성·무결성·가용성**입니다. 확장성은 시스템 설계 품질 속성이자 보안 요소가 아닙니다.

함정 '~성'으로 끝나는 익숙한 IT 용어를 하나 섞어 넣는 방식입니다. 세 개를 정확히 알고 있으면 즉시 걸러집니다.

강사 한마디 각 요소가 **어떤 공격에 대응하는지**까지 엮으면 응용 문제도 풀립니다 — **기밀성 ↔ 도청·유출**(대응: 암호화, 접근통제), **무결성 ↔ 위·변조**(대응: 해시, 전자서명), **가용성 ↔ DDoS·서비스 마비**(대응: 이중화, 백업). 지문에 'DDoS로 서비스가 중단됐다'가 나오면 침해된 것은 **가용성**입니다.

86 접근통제 정책 중, 데이터 소유자가 직접 다른 사용자에게 접근 권한을 부여하는 방식은? 접근통제

- ① MAC(강제적 접근통제)
- ② RBAC(역할기반 접근통제)
- ③ **DAC(임의적 접근통제)**
- ④ ACL

정답 ③

정답 근거 **DAC**는 소유자의 **재량(Discretionary)**으로 권한을 주고받는 방식입니다. 파일 소유자가 공유 설정을 바꾸는 것이 대표 예입니다.

함정 ①MAC은 **관리자가 보안 등급에 따라 강제로** 결정하는 방식(군사·기밀 환경)이라 정반대입니다. D와 M의 의미를 기억하지 않으면 헷갈립니다.

강사 한마디 머리글자의 원어로 구분하면 절대 안 헷갈립니다 — **D는 Discretionary(재량)이니 소유자 마음, M은 Mandatory(강제)이니 규정대로, R은 Role(역할)이니 직책 기준.** 실무에서 가장 널리 쓰이는 건 **RBAC**인데, 사람이 바뀌어도 역할만 부여하면 되어 관리가 편하기 때문입니다. 이 장점을 묻는 문제도 나옵니다.

87 다음 중 비대칭키(공개키) 암호 알고리즘은? 암호화

- ① AES
- ② SEED
- ③ **RSA**
- ④ ARIA

정답 ③

정답 근거 **RSA**는 공개키·개인키 쌍을 사용하는 대표적인 비대칭키 알고리즘입니다. 큰 수의 소인수분해가 어렵다는 점에 기반합니다.

함정 ①AES(미국 표준) ②SEED(국내 표준) ④ARIA(국내 표준)는 **모두 대칭키**입니다. 이름이 낯설수록 비대칭일 것 같은 인상을 주지만 사실은 반대입니다.

강사 한마디 제가 늘 쓰는 요령은 '**비대칭은 RSA·ECC·디피헬만·DSA** 넷뿐, 나머지는 대칭'입니다. 실제로 시험 보기에 등장하는 대칭키는 **DES·3DES·AES·SEED·ARIA·IDEA·RC4** 정도인데, 이것 다 외우기보다 **비대칭 네 개만 외우고 나머지를 대칭으로 처리**하는 편이 훨씬 효율적입니다.

88 해시 함수에 대한 설명으로 옳지 않은 것은? 암호화

- ① 임의 길이의 입력을 고정 길이의 값으로 변환한다
- ② 같은 입력에 대해 항상 같은 출력을 생성한다
- ③ 출력값으로부터 원래 입력을 복원할 수 있다
- ④ 무결성 검증과 비밀번호 저장에 사용된다

정답 ③

정답 근거 해시는 **일방향(단방향)** 함수라 출력값에서 원래 입력을 **복원할 수 없습니다**. 이 성질 때문에 비밀번호 저장에 적합합니다.

함정 '암호'라는 범주에 함께 묶여 있어 복호화가 될 것처럼 느껴집니다. 하지만 **암호화는 복호화가 되고, 해시는 안 됩니다**.

강사 한마디 셋을 **복호화 가능 여부**로 갈라 두면 정리가 끝납니다 — **대칭키: 복호화 O(같은 키), 비대칭키: 복호화 O(다른 키), 해시: 복호화 X**. 대표 해시는 **MD5-SHA** 계열입니다. 참고로 MD5는 충돌 취약점 때문에 **더 이상 안전하지 않다**고 보며, 이 점을 묻는 문제도 있습니다.

89 전자서명(Digital Signature)이 제공하는 보안 서비스로 거리가 먼 것은? 보안

- ① 무결성
- ② 인증
- ③ 부인 방지
- ④ 기밀성

정답 ④

정답 근거 전자서명은 **송신자의 개인키로 서명**하고 누구나 공개키로 검증하므로, **인증·무결성·부인 방지**를 제공합니다. 그러나 내용 자체를 감추지는 않으므로 **기밀성은 제공하지 않습니다**.

함정 '전자서명'이 보안 기술이니 기밀성도 당연히 줄 것 같지만, 서명은 **'누가 썼고 안 바뀌었다'**를 보장할 뿐 내용을 숨기지 않습니다.

강사 한마디 기밀성이 필요하면 **서명과 암호화를 함께** 써야 합니다. 방향을 정리하면 — **서명은 개인키로 하고 공개키로 검증** (나만 만들 수 있고 누구나 확인), **암호화는 공개키로 하고 개인키로 복호화** (누구나 보낼 수 있고 나만 열람). 이 **키 사용 방향**을 반대로 서술한 보기가 자주 나오니 꼭 확인하세요.

90 입력값 검증이 미흡한 웹 페이지에 악의적인 질의문을 삽입하여 데이터베이스를 조작하는 공격은? 보안공격

- ① XSS
- ② 버퍼 오버플로
- ③ CSRF
- ④ **SQL 인젝션**

정답 ④

정답 근거 SQL 인젝션은 입력란에 ' OR '1'='1 같은 구문을 넣어 인증 우회나 데이터 유출을 일으킵니다. 방어는 입력값 검증과 파라미터 바인딩(Prepared Statement)입니다.

함정 ①XSS도 '삽입' 공격이라 헛갈립니다. 차이는 무엇을 삽입하느냐입니다 — SQL 인젝션은 질의문을 DB에, XSS는 스크립트를 웹 페이지에 삽입합니다.

강사 한마디 공격 유형은 '표적'으로 구분하세요 — SQL 인젝션은 DB를, XSS는 다른 사용자의 브라우저를, CSRF는 사용자의 권한을, 버퍼 오버플로는 메모리를 노립니다. 이 한 줄 표만 있으면 어떻게 서술을 바꿔도 분류됩니다. 참고로 이 넷은 시큐어 코딩 문제에서 대응 방안과 짝으로도 나옵니다.

91 사용자가 자신의 의도와 무관하게, 로그인된 권한을 이용해 공격자가 원하는 요청을 서버에 전송하도록 만드는 공격은? 보안공격

- ① XSS
- ② 스니핑
- ③ 세션 하이재킹
- ④ **CSRF**

정답 ④

정답 근거 CSRF(Cross-Site Request Forgery)는 이미 인증된 사용자의 브라우저가 공격자가 준비한 요청을 자기도 모르게 보내게 만듭니다. 방어는 CSRF 토큰·Referer 검증입니다.

함정 ①XSS와 짝으로 출제되며 가장 많이 혼동됩니다. XSS는 스크립트를 실행시켜 정보를 빼내는 것, CSRF는 요청을 위조해 행위를 시키는 것입니다.

강사 한마디 저는 이렇게 구분합니다 — XSS는 '훔치기', CSRF는 '시키기'. XSS는 쿠키·세션을 탈취하는 데 관심이 있고, CSRF는 탈취 없이 피해자 손을 빌려 행동(송금·비밀번호 변경)하게 만듭니다. 지문에 '의도하지 않은 요청'이 있으면 CSRF, '스크립트 실행'이 있으면 XSS입니다.

92 메모리에 할당된 저장 공간의 경계를 넘어 데이터를 기록하여 프로그램의 제어 흐름을 변경하는 공격은? 보안공격

- ① 버퍼 오버플로
- ② 레이스 컨디션
- ③ 포맷 스트링
- ④ 리버스 엔지니어링

정답 ①

정답 근거 버퍼 오버플로는 정해진 크기를 넘겨 인접 메모리를 덮어써서 **복귀 주소를 조작**하는 등 실행 흐름을 가로칩니다. C의 `strcpy`, `gets` 처럼 **경계 검사가 없는 함수**가 원인입니다.

함정 ②레이스 컨디션은 **동시 접근의 순서 문제**를 악용하는 공격이라 메모리 경계와는 다릅니다.

강사 한마디 방어책까지 세트로 외우면 응용 문제도 잡힙니다 — 버퍼 오버플로 대응은 **경계 검사, 안전한 함수 사용(strncpy), 스택 가드, ASLR**입니다. 그리고 이 공격은 시큐어 코딩에서 **'입력 데이터 검증 및 표현'** 항목에 속합니다. 시큐어 코딩 7개 항목 중 **입력값 검증**이 가장 자주 출제됩니다.

93 침입 탐지 시스템(IDS)과 침입 방지 시스템(IPS)의 차이로 옳은 것은? 보안솔루션

- ① IDS는 탐지와 차단을 모두 수행하고, IPS는 탐지만 수행한다
- ② **IDS는 탐지 후 경고하며, IPS는 탐지와 함께 능동적으로 차단한다**
- ③ 둘은 동일한 기능을 하며 명칭만 다르다
- ④ IDS는 네트워크에만, IPS는 호스트에만 설치할 수 있다

정답 ②

정답 근거 **IDS는 탐지·경보**까지, **IPS는 탐지 후 차단**까지 수행합니다. IPS는 트래픽 경로 위에 놓여 실시간으로 막습니다.

함정 ①은 정확히 뒤집은 서술입니다. D(Detection)와 P(Prevention)의 뜻만 알면 즉시 걸러집니다.

강사 한마디 영어 원어가 곧 답입니다 — **Detection은 발견, Prevention은 예방(차단)**. 함께 정리할 것: **방화벽**은 미리 정한 규칙으로 **통과/차단**을 결정하고, **IDS/IPS**는 **공격 패턴·이상 행위**를 분석합니다. 그리고 **DMZ**는 외부 공개 서버를 두는 **완충 구역**, **VPN**은 공용망 위의 **암호화된 전용 통로**입니다.

94 다수의 감염된 좀비 PC를 이용해 동시에 대량의 트래픽을 발생시켜 서비스를 마비시키는 공격은? 보안공격

① DDoS

② 스니핑

③ 스푸핑

④ APT

정답 ①

정답 근거 DDoS(Distributed Denial of Service)는 분산된 다수의 시스템에서 동시에 공격해 서비스를 가용 불가 상태로 만듭니다.

함정 단일 시스템에서 하면 DoS이고, 여러 대(분산)면 DDoS입니다. '좀비 PC 다수'가 D 하나 더 붙는 근거입니다.

강사 한마디 이 공격이 침해하는 보안 요소가 가용성이라는 연결도 함께 기억하세요 — 이렇게 묻는 문제가 실제로 나옵니다. 그리고 좀비 PC들의 무리를 봇넷(Botnet)이라 하고, 이를 조종하는 서버를 C&C 서버라 부릅니다. 용어 문제로 나올 수 있으니 함께 답아 두세요.

95 네트워크상의 패킷을 몰래 가로채어 훑쳐보는 수동적 공격은? 보안공격

① 스푸핑(Spoofing)

② 스니핑(Sniffing)

③ 피싱(Phishing)

④ 랜섬웨어

정답 ②

정답 근거 스니핑은 전송 중인 패킷을 도청하는 공격으로, 데이터를 변경하지 않으므로 수동적(passive) 공격으로 분류됩니다.

함정 ①스푸핑은 IP·MAC·DNS 등을 위조해 신분을 속이는 능동적 공격입니다. 이름이 비슷해 가장 자주 바뀌어 나옵니다.

강사 한마디 영어 뜻으로 가르면 명확합니다 — sniff는 '냄새를 맡다(엿듣다)', spoof는 '속이다'. 그리고 수동적/능동적 분류도 자주 묻습니다: 수동적=도청·트래픽 분석(탐지가 어려움), 능동적=위조·변조·삽입·서비스 거부. 스니핑의 대응책은 암호화 통신(SSL/TLS)이라는 점까지 챙기면 완벽합니다.

96 시스템의 파일을 암호화한 뒤 복호화 키를 대가로 금전을 요구하는 악성코드는? 악성코드

- ① 트로이 목마
- ② 랜섬웨어
- ③ 웜(Worm)
- ④ 스파이웨어

정답 ②

정답 근거 랜섬웨어(Ransom + Ware)는 몸값을 요구하는 악성코드입니다. 가장 효과적인 대비책은 정기적인 오프라인 백업입니다.

함정 ③웜은 스스로 복제·전파하는 것이 특징이고, ①트로이 목마는 정상 프로그램으로 위장해 잠입합니다. 각각의 정의가 다릅니다.

강사 한마디 악성코드는 '행동 방식'으로 나뉩니다 — 바이러스=숙주 파일에 기생, 웜=스스로 복제·전파(숙주 불필요), 트로이 목마=위장·자기복제 없음, 랜섬웨어=암호화·금전 요구, 스파이웨어=정보 수집. 특히 '웜은 자기 복제, 트로이 목마는 자기 복제를 하지 않는다'는 비교가 시험에 자주 나옵니다.

97 특정 대상을 목표로 장기간에 걸쳐 은밀하고 지속적으로 수행되는 지능형 표적 공격은? 보안

- ① 제로데이 공격
- ② APT
- ③ 무차별 대입 공격
- ④ 사회공학

정답 ②

정답 근거 APT(Advanced Persistent Threat)는 특정 조직을 노려 오랜 기간 잠복하며 목표를 달성할 때까지 공격을 이어갑니다.

함정 ①제로데이는 패치가 나오기 전의 취약점을 노리는 공격으로, 기간이 아니라 시점에 초점이 있습니다.

강사 한마디 머리글자가 곧 정의입니다 — Advanced(지능적) Persistent(지속적) Threat(위협). 지문에 '장기간·지속적·특정 대상'이 있으면 APT입니다. 참고로 APT 공격은 보통 사회공학(스피어 피싱)으로 시작해 내부로 침투하는 흐름을 갖는데, 이 과정을 묻는 문제도 출제됩니다.

98 거래 내역을 블록에 담아 체인으로 연결하고, 이를 참여자 모두가 나눠 보관하여 위·변조를 어렵게 만든 기술은? 신기술

솔

① 블록체인

- ② 빅데이터
- ③ 클라우드 컴퓨팅
- ④ 엣지 컴퓨팅

정답 ①

정답 근거 블록체인은 분산 원장 기술로, 모든 참여자가 동일한 장부를 갖기 때문에 일부를 조작해도 다수의 사본과 불일치하여 위·변조가 사실상 불가능합니다.

함정 신기술 용어는 정의가 비슷하게 느껴져 헷갈립니다. 블록체인의 결정적 단서는 '분산 저장'과 '위·변조 방지'입니다.

강사 한마디 5과목 신기술 용어는 매년 새 단어가 나오는 영역이라 다 외울 수 없습니다. 저는 영어 단어의 뜻으로 유추하는 훈련을 권합니다 — Block+Chain(블록을 사슬로), Edge(가장자리=단말 가까이서 처리), Docker(컨테이너), Mesh(그물망). 모르는 용어가 나와도 단어 뜻에서 절반은 맞힐 수 있습니다. 그래도 모르면 과감히 넘기세요 — 5과목은 과락만 면하면 됩니다.

99 애플리케이션을 격리된 컨테이너 단위로 패키징하여 환경에 상관없이 동일하게 실행되도록 하는 기술은? 신기술

- ① 가상 머신(VM)
- ② 도커(Docker)
- ③ 하둡(Hadoop)
- ④ 쿠버네티스 클러스터의 노드

정답 ②

정답 근거 도커는 애플리케이션과 실행 환경을 컨테이너로 묶어 어디서든 같은 방식으로 동작하게 합니다. OS 커널을 공유해 VM 보다 가볍고 빠릅니다.

함정 ①가상 머신도 격리 기술이지만, VM은 게스트 OS 전체를 포함해 무겁습니다. '가볍다·OS를 공유한다'가 컨테이너의 표식입니다.

강사 한마디 둘의 차이는 OS 포함 여부 하나로 같습니다 — VM은 OS까지 통째로(무거움), 컨테이너는 앱만(가벼움). 그리고 컨테이너가 많아지면 관리 도구가 필요한데 그게 쿠버네티스(K8s)입니다. '컨테이너 오케스트레이션'이라는 표현이 나오면 쿠버네티스입니다. 이 세 용어는 묶어서 나오는 편입니다.

100 시큐어 코딩 관점에서 가장 적절하지 않은 것은? 시큐어코딩

- ① 사용자 입력값은 서버 측에서 반드시 검증한다
- ② 비밀번호는 해시하여 저장한다
- ③ 오류 메시지에 시스템 내부 정보를 상세히 노출한다
- ④ 권한 검사는 모든 요청마다 수행한다

정답 ③

정답 근거 오류 메시지에 DB 구조·경로·쿼리문 등이 노출되면 공격자에게 정보를 제공합니다. 사용자에게는 일반적인 메시지만 보이고 상세 내용은 서버 로그에만 남겨야 합니다.

함정 디버깅에 유리하다는 이유로 상세 메시지를 좋게 볼 수 있지만, 운영 환경에서는 취약점입니다. 개발 편의와 보안이 충돌하는 지점입니다.

강사 한마디 시큐어 코딩 문제는 '공격자에게 유리한 선택지가 답'이라는 원칙으로 풀립니다. 정보를 더 보여주거나, 검증을 생략하거나, 클라이언트만 믿는 보기가 있으면 그게 오답(=정답)입니다. 특히 '클라이언트 측에서만 검증한다'는 보기는 무조건 틀린 것입니다 — 클라이언트 검증은 우회가 가능하기 때문입니다.

① 결합도 · 응집도 판별

101 모듈 A가 모듈 B에게 구조체(레코드) 전체를 넘겼지만, B는 그중 한 필드만 사용한다. 이때의 결합도는? 결합도

- ① 자료 결합도
- ② 스탬프 결합도
- ③ 제어 결합도
- ④ 공통 결합도

정답 ②

정답 근거 자료구조 전체를 넘기고 일부만 쓰면 스탬프(Stamp) 결합도입니다. 쓰지 않는 필드까지 알게 되어 자료 결합보다 결합이 강합니다.

함정 '값을 넘겼다'만 보고 자료 결합도로 가면 틀립니다. 넘긴 것이 날개 값인지 덩어리인지가 갈림길입니다.

강사 한마디 판별 순서를 고정하세요 — 전역변수?(공통) → 내부 직접 참조?(내용) → 넘긴 게 지시·플래그?(제어) → 구조체 통째?(스탬프) → 날개 값?(자료). 위에서부터 훑으면 어떤 상황이든 3초에 분류됩니다.

102 모듈 A가 모듈 B의 내부 변수를 직접 참조하여 값을 변경한다. 가장 바람직하지 않은 이 결합도는? 결합도

- ① 공통 결합도
- ② 외부 결합도
- ③ 내용 결합도
- ④ 제어 결합도

정답 ③

정답 근거 다른 모듈의 **내부를 직접** 건드리면 **내용(Content) 결합도**로, 6가지 중 가장 강한(나쁜) 결합입니다.

함정 공통 결합도와 헷갈립니다. 공통은 **전역변수를 공유**하는 것이고, 내용은 **상대 모듈 안으로 침범**하는 것입니다.

강사 한마디 저는 '남의 집에 들어가느냐'로 구분합니다. **공용 창고를 같이 쓰면 공통**, **남의 집 안방을 뒤흔들면 내용**. 내용 결합은 상대를 고치면 무조건 같이 깨지므로 최악입니다.

103 모듈 내부의 요소들이 모두 하나의 기능을 수행하기 위해서만 존재한다. 이 응집도는? 응집도

- ① 순차적 응집도
- ② 교환적 응집도
- ③ 절차적 응집도
- ④ 기능적 응집도

정답 ④

정답 근거 단일 기능만 수행하면 **기능적(Functional) 응집도**로 가장 이상적입니다.

함정 ①순차적은 **앞의 출력이 뒤의 입력**이 되는 경우로, 여러 작업이 이어진 상태입니다. '하나의 기능'과는 다릅니다.

강사 한마디 응집도는 **기·순·교·절·시·논·우** 순서(강→약)를 외우고, 지문에서 **묶인 기준**을 찾으세요. 기준이 '단일 기능'이면 최상위입니다. 좋은 설계는 **결합도 낮게, 응집도 높게**인데, 둘 다 압기 순서의 **앞쪽이 좋다**는 점이 편합니다.

104 입력 자료를 읽어 가공한 결과를 다음 요소가 그대로 입력으로 받아 처리하는 모듈의 응집도는? 응집도

- ① 순차적 응집도
- ② 시간적 응집도
- ③ 논리적 응집도
- ④ 우연적 응집도

정답 ①

정답 근거 앞 요소의 **출력**이 뒤 요소의 **입력**이 되면 **순차적(Sequential) 응집도**입니다. 기능적 다음으로 좋은 응집도입니다.

함정 ③절차적 응집도와 비슷해 보입니다. 절차적은 **정해진 순서로 실행**될 뿐 데이터가 이어지지는 않습니다. **데이터가 이어지면 순차적**입니다.

강사 한마디 둘의 차이는 '**데이터가 흐르느냐, 순서만 있느냐**'입니다. 컨베이어 벨트처럼 결과물이 넘어가면 순차적, 그냥 순서대로 하는 일이면 절차적. 이 구분을 물으면 대부분 순차적이 답입니다.

105 바람직한 모듈 설계 방향으로 옳은 것은? 모듈화

- ① 결합도는 높이고 응집도는 낮춘다
- ② 결합도와 응집도 모두 높인다
- ③ **결합도는 낮추고 응집도는 높인다**
- ④ 결합도와 응집도 모두 낮춘다

정답 ③

정답 근거 모듈은 **서로 독립적(결합도 ↓)**이고 **내부는 한 가지 일에 집중(응집도 ↑)**해야 좋습니다.

함정 두 지표의 방향이 반대라 헷갈립니다. '둘 다 높게/낮게'라는 보기가 항상 함께 나옵니다.

강사 한마디 한 문장으로 '**남과는 멀게, 안은 뽕뽕하게**'라고 외우세요. 여기에 **팬인은 높게, 팬아웃은 낮게**까지 붙이면 모듈 설계 문제는 전부 방어됩니다.

② 테스트 — 스텝·드라이버 / 화이트·블랙박스

106 하향식 통합 테스트에서 아직 구현되지 않은 하위 모듈을 대신하는 임시 모듈은? 통합테스트

- ① 드라이버(Driver)
- ② 스텝(Stub)
- ③ 목(Mock)
- ④ 하네스(Harness)

정답 ②

정답 근거 하향식은 위에서 아래로 내려가므로 **없는 하위**를 **스텝**으로 대체합니다.

함정 드라이버와 반드시 구분하세요. **상향식**일 때 없는 **상위**를 대신하는 것이 드라이버입니다.

강사 한마디 '없는 쪽을 채운다'로 기억하면 절대 안 헛갈립니다. 하향식→아래가 없음→스텝, 상향식→위가 없음→드라이버. 어원으로도 **드라이버=운전자(위에서 조종)**, **스텝=그루터기(아래 밀동)**.

107 상향식 통합 테스트에 대한 설명으로 옳은 것은? 통합테스트

- ① 상위 모듈부터 검증하며 스텝이 필요하다
- ② 하위 모듈부터 검증하며 드라이버가 필요하다
- ③ 모든 모듈을 한 번에 결합해 검증한다
- ④ 보조 모듈 없이 수행할 수 있다

정답 ②

정답 근거 상향식(Bottom-Up)은 **하위 모듈부터** 검증하고, 이를 호출해 줄 **드라이버**가 필요합니다.

함정 ③은 **빅뱅(Big-Bang)** 방식의 설명입니다. 빅뱅은 보조 모듈이 필요 없는 대신 **결합 위치를 찾기 어렵습니다**.

강사 한마디 통합 방식은 세 가지 — **하향식(스텝)·상향식(드라이버)·빅뱅(둘 다 없음)**. 빅뱅의 단점(원인 추적 곤란)까지 세트로 물어보니 함께 기억하세요.

108 다음 중 화이트박스 테스트 기법은? 테스트기법

- ① 동등 분할
- ② 경계값 분석
- ③ 기초 경로 검사
- ④ 원인-결과 그래프

정답 ③

정답 근거 기초 경로 검사(Basis Path Testing)는 프로그램의 제어 흐름·경로를 분석하는 화이트박스 기법입니다. 순환 복잡도와 함께 다룹니다.

함정 나머지 셋은 전부 블랙박스입니다. 목록을 반대로 알고 있으면 통째로 틀립니다.

강사 한마디 두 목록을 통으로 외우는 게 가장 빠릅니다. 화이트박스 = 커버리지(구문·결정·조건·다중조건) + 기초 경로 + 루프 검사. 블랙박스 = 동등분할·경계값·원인결과·결정표·상태전이·오류추정. 한쪽 목록에 없는 단어가 섞여 있으면 그게 답입니다.

109 프로그램의 내부 구조를 모르는 상태에서 입력과 출력만으로 기능을 검증하는 테스트는? 테스트기법

- ① 화이트박스 테스트
- ② 블랙박스 테스트
- ③ 구조 기반 테스트
- ④ 경로 테스트

정답 ②

정답 근거 내부를 보지 않고 명세(입력→출력) 기준으로 검증하면 블랙박스입니다.

함정 ③구조 기반 테스트는 화이트박스의 다른 이름입니다. 용어를 바꿔 부르는 방식으로 함정을 만듭니다.

강사 한마디 별칭까지 알아두면 안전합니다 — 화이트박스 = 구조 기반 = 코드 기반, 블랙박스 = 명세 기반 = 기능 기반. 시험은 이 별칭을 섞어 씁니다.

110 검증 기준이 약한 것부터 강한 순서로 바르게 나열한 것은? 커버리지

- ① 구문 → 결정 → 조건 → 다중조건
- ② 조건 → 구문 → 결정 → 다중조건
- ③ 다중조건 → 조건 → 결정 → 구문
- ④ 결정 → 구문 → 조건 → 다중조건

정답 ①

정답 근거 구문(문장) → 결정(분기) → 조건 → 조건/결정 → 다중조건 순으로 검증 강도가 높아집니다.

함정 ③은 정확히 역순입니다. '강한 것부터'인지 '약한 것부터'인지 **질문 방향**을 먼저 확인하세요.

강사 한마디 의미로 이해하면 순서가 저절로 나옵니다 — 구문은 **줄만 실행**하면 되고, 결정은 **참·거짓 두 갈래**를 다 타야 하고, 다중조건은 **조건들의 모든 조합**을 봐야 하니 가장 뻑뻑합니다. 요구가 많아질수록 강한 기준입니다.

③ UML — 집합·합성 / include·extend

111 UML에서 속이 빈 마름모(◇)로 표현하며, 부분 객체가 전체 객체 없이도 존재할 수 있는 관계는? UML

- ① 합성(Composition)
- ② **집합(Aggregation)**
- ③ 일반화(Generalization)
- ④ 실체화(Realization)

정답 ②

정답 근거 **집합(Aggregation)**은 전체-부분이지만 부분이 **독립적으로 존재**할 수 있습니다. 기호는 **빈 마름모** ◇입니다.

함정 합성은 **채워진 마름모** ◆이고 생명주기를 공유합니다. 마름모의 **속이 비었나 찼나**가 결정적입니다.

강사 한마디 '빈 마름모는 헤어져도 살고, 찬 마름모는 같이 죽는다'로 외우세요. 부서-사원은 집합(이직 가능), 주문-주문상세는 합성(주문 취소되면 함께 소멸).

112 유스케이스에서 특정 조건이 만족될 때만 선택적으로 수행되는 기능을 표현하는 관계는? UML

- ① include
- ② **extend**
- ③ generalization
- ④ dependency

정답 ②

정답 근거 **extend**는 조건부·선택적 확장입니다. '쿠폰 적용', '추가 옵션 선택'처럼 항상 일어나지 않는 기능에 씁니다.

함정 include는 **반드시 항상** 포함되는 필수 관계입니다. 정확히 반대 개념입니다.

강사 한마디 부사만 보면 끝납니다 – '반드시·항상·공통'이면 include, '~인 경우·선택적으로·필요하면'이면 extend. 지문에 서 이 단어에 동그라미 치는 습관을 들이세요.

113 UML 다이어그램 중 동적(행위) 다이어그램으로만 묶인 것은? UML

- ① 클래스, 객체, 패키지
- ② **유스케이스, 시퀀스, 상태**
- ③ 컴포넌트, 배치, 복합체구조
- ④ 클래스, 활동, 배치

정답 ②

정답 근거 유스케이스·시퀀스·상태·활동·커뮤니케이션·타이밍이 **행위(동적)** 다이어그램입니다.

함정 ④처럼 동적 하나(활동)에 정적 둘을 섞어 놓는 방식이 혼합니다. 보기 안의 셋을 모두 확인해야 합니다.

강사 한마디 묶음 문제는 **확실한 하나를 찾아 그 보기를 통째로 버리는** 소거법이 빠릅니다. 클래스·배치·컴포넌트·패키지·객체가 보이면 그 보기는 즉시 탈락(정적).

114 자식 클래스가 부모 클래스를 상속받는 관계를 나타내는 UML 표기는? UML

- ① **실선 + 속 빈 삼각형**
- ② 점선 + 화살표
- ③ 실선 + 채워진 마름모
- ④ 점선 + 속 빈 삼각형

정답 ①

정답 근거 **일반화(상속)**는 실선에 속 빈 삼각형으로, 자식에서 부모를 향해 그립니다.

함정 ④는 **실체화(인터페이스 구현)** 표기입니다. 삼각형은 같고 선이 실선이나 점선이나로 갈립니다.

강사 한마디 선의 종류에 의미가 있습니다 – 실선은 '진짜 관계', 점선은 '느슨한 관계'. 그래서 상속은 실선, 인터페이스 구현·의존은 점선입니다. 이 원리를 알면 표기를 외우지 않아도 추론됩니다.

115 시퀀스 다이어그램의 구성 요소가 아닌 것은? UML

- ① 생명선(Lifeline)
- ② 활성화 박스(Activation)
- ③ 메시지(Message)
- ④ 마름모(Diamond)

정답 ④

정답 근거 시퀀스 다이어그램은 객체·생명선·활성박스·메시지로 구성됩니다. 마름모는 클래스 다이어그램의 집합·합성 기호입니다.

함정 UML이라는 큰 범주 안에서 다른 다이어그램의 기호를 섞어 놓는 방식입니다.

강사 한마디 시퀀스는 '시간이 아래로 흐르는 그림'이라고 떠올리세요. 세로선(생명선) 위를 가로 화살표(메시지)가 오가는 구조라, 마름모 같은 구조 기호가 들어갈 자리가 없습니다.

④ 데이터베이스 — 정규화 · SQL 삭제 3형제

116 릴레이션의 모든 속성이 원자값만 갖도록 하는 정규형은? 정규화

- ① 제1정규형
- ② 제2정규형
- ③ 제3정규형
- ④ BCNF

정답 ①

정답 근거 한 칸에 여러 값(반복 그룹)이 들어가지 않도록 하는 것이 제1정규형(1NF)입니다. 정규화의 출발점입니다.

함정 '속성'이라는 말 때문에 2NF(부분 종속)와 헷갈립니다. 값의 형태를 말하면 1NF, 종속 관계를 말하면 2NF 이상입니다.

강사 한마디 제거 대상만 외우면 끝납니다 — 1NF는 반복(비원자값), 2NF는 부분 종속, 3NF는 이행 종속, BCNF는 결정자. 앞 글자로 도·부·이·결. 지문의 키워드가 그대로 답을 가리킵니다.

117 릴레이션 R에서 $A \rightarrow B$, $B \rightarrow C$ 의 함수 종속이 존재할 때 제거해야 하는 종속과 그 정규형은? 정규화

- ① 부분 함수 종속, 2NF
- ② 이행 함수 종속, 3NF
- ③ 다치 종속, 4NF
- ④ 조인 종속, 5NF

정답 ②

정답 근거 A를 통해 C가 간접적으로 결정되는 **이행(Transitive) 함수 종속**이며, 이를 제거하면 **제3정규형**이 됩니다.

함정 부분 종속(2NF)과 혼동됩니다. 부분 종속은 **복합키의 일부**에만 종속되는 것으로, 키가 두 개 이상일 때만 생깁니다.

강사 한마디 모양으로 구분하세요 — $A \rightarrow B \rightarrow C$ 처럼 징검다리면 이행(3NF), (A,B)가 키인데 A만으로 C가 결정되면 부분(2NF). 화살표를 그려 보면 즉시 갑니다.

118 WHERE 절과 HAVING 절에 대한 설명으로 옳은 것은? SQL

- ① WHERE는 그룹화 후, HAVING은 그룹화 전에 적용된다
- ② WHERE에는 집계 함수를 조건으로 쓸 수 있다
- ③ **HAVING은 GROUP BY로 묶인 그룹에 조건을 적용한다**
- ④ 둘은 완전히 동일하게 동작한다

정답 ③

정답 근거 **HAVING은 그룹화 후 그룹에** 조건을 겁니다. `HAVING COUNT(*) ≥ 2` 처럼 집계 함수를 쓸 수 있습니다.

함정 ②가 매력적입니다. WHERE는 그룹화 **전**에 실행되므로 그 시점에는 COUNT 값이 없어 **집계 함수를 쓸 수 없습니다**.

강사 한마디 실행 순서를 알면 전부 설명됩니다 — **FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY**. 이 한 줄이면 '별칭을 WHERE에서 못 쓰는 이유'까지 함께 풀립니다.

119 다음 중 롤백(ROLLBACK)이 가능한 삭제 명령은? SQL

- ① DROP TABLE
- ② TRUNCATE TABLE
- ③ **DELETE FROM**
- ④ ALTER TABLE

정답 ③

정답 근거 **DELETE**는 DML이므로 트랜잭션 안에서 **ROLLBACK**으로 되돌릴 수 있습니다. 조건(WHERE)도 줄 수 있습니다.

함정 TRUNCATE도 '행을 지운다'는 점이 같지만 **DDL**이라 자동 커밋되어 되돌릴 수 없습니다.

강사 한마디 삭제 3형제를 표로 굳히세요 — **DELETE(DML·조건O·롤백O·느림)**, **TRUNCATE(DDL·전체·롤백X·빠름)**, **DROP(DDL·테이블 자체 삭제)**. 문제에 조건이 둘 이상 붙으면 이 표에서 하나로 좁혀집니다.

120 트랜잭션의 ACID 특성과 설명이 잘못 짝지어진 것은? 트랜잭션

- ① 원자성 — 전부 수행되거나 전혀 수행되지 않는다
- ② 일관성 — 실행 전후 데이터베이스가 일관된 상태를 유지한다
- ③ 격리성 — 동시에 실행되는 트랜잭션끼리 서로 영향을 주지 않는다
- ④ **지속성 — 트랜잭션 수행 중 언제든지 취소할 수 있다**

정답 ④

정답 근거 **지속성(Durability)**은 완료된 결과가 **영구히 보존**된다는 뜻입니다. '언제든 취소'는 오히려 지속성과 반대입니다.

함정 '취소'가 원자성의 롤백을 연상시켜 그럴듯해 보입니다. 하지만 지속성은 **완료 후**의 성질입니다.

강사 한마디 시점으로 나누면 명확합니다 — **실행 중: 원자성·격리성, 완료 후: 지속성, 전후 비교: 일관성**. 지문에 '장애가 나도 영구히'가 있으면 지속성입니다.

⑤ 보안 — 암호 분류 / XSS·CSRF / IDS·IPS

121 암호화와 복호화에 서로 다른 키를 사용하는 알고리즘으로만 묶인 것은? 암호화

- ① AES, DES
- ② **RSA, ECC**
- ③ SEED, ARIA
- ④ MD5, SHA

정답 ②

정답 근거 **RSA·ECC**가 공개키/개인키 쌍을 쓰는 **비대칭키** 알고리즘입니다.

함정 ④MD5·SHA는 **해시**라서 애초에 복호화가 없습니다. '다른 키'라는 조건에 해당하지 않습니다.

강사 한마디 **비대칭**은 **RSA·ECC·디피헬만·DSA** 넷뿐이라고 외우고 나머지는 대칭으로 처리하세요. 실제 시험 보기의 90%가 이 방법으로 해결됩니다. 해시(MD5·SHA)는 **복호화 불가**라 따로 분류합니다.

122 대칭키 암호 방식의 특징으로 옳은 것은? 암호화

- ① 키 분배가 쉽고 관리할 키의 수가 적다
- ② 비대칭키보다 암호·복호화 속도가 빠르다
- ③ 공개키와 개인키 쌍을 사용한다
- ④ 전자서명에 주로 사용된다

정답 ②

정답 근거 대칭키는 연산이 단순해 속도가 빠릅니다. 그래서 대용량 데이터 암호화에 씁니다.

함정 ①이 함정입니다. 대칭키는 상대방마다 다른 키가 필요해 사람이 n명이면 키가 $n(n-1)/2$ 개로 폭증하고, 키 분배도 어렵습니다.

강사 한마디 둘은 맞바꿈 관계입니다 — 대칭키: 빠르지만 키 분배·관리가 어려움, 비대칭키: 느리지만 키 분배가 쉽고 전자서명 가능. 그래서 실무는 키 교환은 비대칭, 데이터는 대칭으로 섞어 씁니다(하이브리드).

123 공격자가 게시판에 스크립트를 삽입해 이를 열람한 다른 사용자의 브라우저에서 실행되게 하는 공격은? 공격

- ① SQL 인젝션
- ② XSS
- ③ CSRF
- ④ 세션 하이재킹

정답 ②

정답 근거 XSS(Cross-Site Scripting)는 웹 페이지에 악성 스크립트를 심어 열람자의 브라우저에서 실행시켜 쿠키·세션을 탈취합니다.

함정 CSRF와 최다 혼동 짝입니다. CSRF는 스크립트 실행이 아니라 피해자가 의도치 않은 요청을 보내게 만드는 공격입니다.

강사 한마디 XSS는 '훔치기', CSRF는 '시키기'로 갈라 두세요. 지문에 '스크립트'가 있으면 XSS, '의도하지 않은 요청'이 있으면 CSRF입니다. 이 한 줄로 두 문제를 동시에 잡습니다.

124 침입을 탐지한 뒤 경고만 발생시키고 직접 차단하지는 않는 보안 시스템은? 보안장비

- ① IDS
- ② IPS
- ③ 방화벽
- ④ WAF

정답 ①

정답 근거 IDS(Intrusion Detection System)는 탐지·경보까지가 역할입니다.

함정 IPS와 한 글자 차이로 매번 바뀌 냅니다. P는 Prevention(차단까지)입니다.

강사 한마디 영어 원어가 곧 답입니다 — D=Detection(발견만), P=Prevention(막기까지). 함께 정리하면 방화벽은 규칙 기반 통과/차단, WAF는 웹 전용 방화벽입니다.

125 네트워크 패킷을 몰래 도청하는 수동적 공격과, 주소·신분을 위조하는 능동적 공격을 바르게 짝지은 것은? 공격

- ① 스푸핑 — 스니핑
- ② 스니핑 — 스푸핑
- ③ 피싱 — 파밍
- ④ 스미싱 — 스니핑

정답 ②

정답 근거 스니핑은 엿듣는 수동적 공격, 스푸핑은 속이는 능동적 공격입니다.

함정 ①처럼 순서를 뒤집은 보기가 항상 함께 나옵니다. 이름이 비슷해 순간적으로 헷갈립니다.

강사 한마디 영어 뜻으로 못 박으세요 — sniff = 냄새 맡다(엿듣다), spoof = 속이다. 그리고 분류도 함께: 수동적 = 도청·트래픽 분석(탐지 어려움), 능동적 = 위조·변조·삽입·서비스 거부.

⑥ 계산 문제 총집합

126 제어 흐름 그래프의 노드가 9개, 간선이 13개일 때 순환 복잡도 $V(G)$ 는? 계산

- ① 4
- ② 5
- ③ 6
- ④ 7

정답 ③

정답 근거 $V(G) = E - N + 2 = 13 - 9 + 2 = 6$ 입니다.

함정 E와 N의 순서를 바꿔 $9 - 13 + 2 = -2$ 가 되면 당황합니다. 간선이 항상 앞입니다.

강사 한마디 결과가 음수나 0이면 무조건 계산 실수입니다. 계산 기준으로 쓰세요. 코드를 주고 물으면 조건문 개수 + 1로 더 빠르게 구할 수 있습니다.

127 HRN 방식에서 대기 시간 9, 서비스 시간 3인 작업의 우선순위 값은? 계산

- ① 3
- ② 4
- ③ 6
- ④ 12

정답 ②

정답 근거 (대기 + 서비스) ÷ 서비스 = $(9 + 3) ÷ 3 = 12 ÷ 3 = 4$ 입니다.

함정 분자에 서비스 시간을 더하는 것을 잊고 $9 ÷ 3 = 3$ (①)으로 계산하는 실수가 가장 흔합니다.

강사 한마디 결과는 항상 1보다 큼니다(대기가 0이어도 1). 1 이하가 나오면 계산이 틀린 겁니다. 공식을 잊었다면 'SJF의 기아를 막으려 기다린 시간에 가산점을 준다'는 취지에서 복원하세요.

128 페이지 참조열이 2 3 2 1 5 2 4이고 프레임이 3개일 때, FIFO 알고리즘의 페이지 부재 횟수는? 계산

- ① 4회
- ② 5회
- ③ 6회
- ④ 7회

정답 ③

정답 근거 2(부재) 3(부재) 2(적중) 1(부재) → [2,3,1]. 5(부재, 가장 먼저 들어온 2 교체) → [3,1,5]. 2(부재) → [1,5,2]. 4(부재) → 총 6회입니다.

함정 LRU로 풀면 5회가 나와 ②를 고르게 됩니다. FIFO는 사용 여부와 무관하게 들어온 순서로만 내보냅니다.

강사 한마디 FIFO의 핵심은 적중해도 순서가 바뀌지 않는다는 점입니다. 위에서 2가 중간에 적중했지만 FIFO는 그걸 무시하고 먼저 내보내죠 — 그래서 LRU보다 부재가 1회 더 납니다. 표를 그려 프레임 상태를 매 단계 적는 것이 정석입니다.

129 서브넷 마스크가 /27인 네트워크에서 할당 가능한 호스트 수는? 계산

- ① 14개
- ② 30개
- ③ 62개
- ④ 32개

정답 ②

정답 근거 호스트 비트 = $32 - 27 = 5$ 비트 → $2^5 = 32$, 여기서 네트워크·브로드캐스트 2개를 빼면 30개입니다.

함정 ④32는 -2를 하지 않은 값입니다. 서브넷 문제의 최대 실수 요인입니다.

강사 한마디 자주 나오는 값은 통째로 외우세요 — /24=254, /25=126, /26=62, /27=30, /28=14. 프리픽스가 1 늘 때마다 대략 절반이 되는 규칙만 알면 계산 없이 즉답합니다.

130 LOC 기법에서 낙관치 6,000 · 기대치 15,000 · 비관치 18,000 라인일 때 예측 LOC는? 계산

- ① 13,000
- ② **14,000**
- ③ 15,000
- ④ 16,000

정답 ②

정답 근거 (낙관 + 4×기대 + 비관) ÷ 6 = (6,000 + 60,000 + 18,000) ÷ 6 = 84,000 ÷ 6 = **14,000**입니다.

함정 단순 평균(13,000)을 답으로 고르기 쉽습니다. **기대치에 4를 곱하는** 가중치가 핵심입니다.

강사 한마디 이 공식은 **PERT의 기대 소요시간 공식과 완전히 같습니다** — 비용 산정과 일정 산정 두 곳에서 동시에 쓰이니 한번 외워 두 배로 씁니다. 계산할 때 **4를 곱했는지**만 확인하면 실수가 없습니다.